PERSONAL REFERENCE · LLM BUILD WEEK

Teach-You-An-LLM Complete Tutorial

From first-principles word vectors to a running nanochat transformer.
Consolidated revision reference — Phases 1 · 2 · 3.1 complete.

Compiled from project sources on 2026-04-24.
Colour key: `[PT] PyTorch built-in` `[NC] nanochat custom`

Contents

PART 5 · **Appendix — Learning Journal: Phase 1 takeaways**

1. BPE implementation walkthrough
2. tiktoken and prepare.py
3. vocab_size handoff

PART I

LLM Fundamentals

From representing words as numbers to the doorstep of attention. Embeddings, Word2Vec, RNNs, LSTMs, encoder–decoders — and why each failed at what the next one fixed.

> Start here to build the vocabulary (literally and mentally) you'll rely on in every later section. Every later idea — Q/K/V, causal masking, residual connections — is a direct descendant of something in this chapter.

From Embeddings to Transformers

Table of Contents

1. Foundations: Representing Words as Numbers

Why We Need Numerical Representations

Neural networks can only process numbers. Text must be converted to vectors.

The Core Problem:

```
"cat" → ??? → [0.5, 0.8, 0.3, 0.2]
"dog" → ??? → [0.4, 0.7, 0.4, 0.3]  ← Should be similar to cat!
"car" → ??? → [0.1, 0.2, 0.9, 0.8]  ← Should be different from cat!
```

Column Vectors: The Standard Representation

An embedding is stored as a **column vector**:

```
"cat" embedding (4 dimensions):
[0.2]  ← Dimension 1
[0.8]  ← Dimension 2
[0.3]  ← Dimension 3
[0.5]  ← Dimension 4
```

Why column vectors? Matrix multiplication flows naturally:

```
Weight matrix × Input vector = Output vector
(3×4)         × (4×1)       = (3×1)
```

Important: Each element (not row) represents a learnable dimension. We don't know what each dimension means - the model learns this during training!

The Embedding Matrix

All embeddings are stored in one large matrix:

```
Embedding Matrix (Vocabulary size × Embedding dimension)
= (50,000 × 768)

            Dim1  Dim2  Dim3  ...  Dim768
Token 0:    0.2   0.5   0.1   ...   0.3   ← Row 0 = token 0's embedding
Token 1:    0.8   0.3   0.4   ...   0.7   ← Row 1 = token 1's embedding
...
Token 2653: 0.25  0.47  0.38  ...   0.51  ← Row 2653 = "bank"'s embedding
...
```

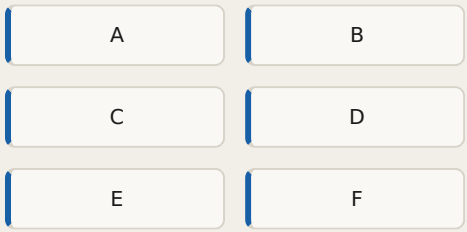
Lookup:

```
token_id = 2653  # "bank"
embedding = embedding_matrix[2653, :]
# → [0.25, 0.47, 0.38, 0.51, ...]
```

---

2. Tokenization

The Pipeline



FLOW

A → B

B → C

C → D

D → E

E → F

What Is a Token?

Tokens are NOT always whole words. Modern tokenizers use **subword tokenization**:

```
"unhappiness" → ["un", "happiness"]
"apologizing" → ["apolog", "izing"]
"600"         → ["6", "0", "0"]  (StarCoder2 - digit per token)
"870"         → ["870"]          (GPT-2 - single token)
```

Vocabulary Sizes

MODEL	VOCABULARY SIZE
BERT	30,522
GPT-2	50,257
GPT-4	~100,000

Larger vocabulary = fewer tokens per word = more text fits in context window

Special Tokens

```
<s>             ← Start of sequence
</s>            ← End of sequence
<|endoftext|>   ← GPT end marker
[CLS]           ← BERT classification token
[SEP]           ← BERT separator
[MASK]          ← BERT masked token
<|user|>        ← Chat role markers (Phi-3)
<|assistant|>   ← Chat role markers
```

Impact of Vocabulary on Embedding Matrix

```
GPT-2: 50,257 × 768  = 38,597,376 parameters
GPT-4: 100,000 × 768 = 76,800,000 parameters
```

Larger vocabulary = bigger embedding matrix = more parameters just for embeddings!

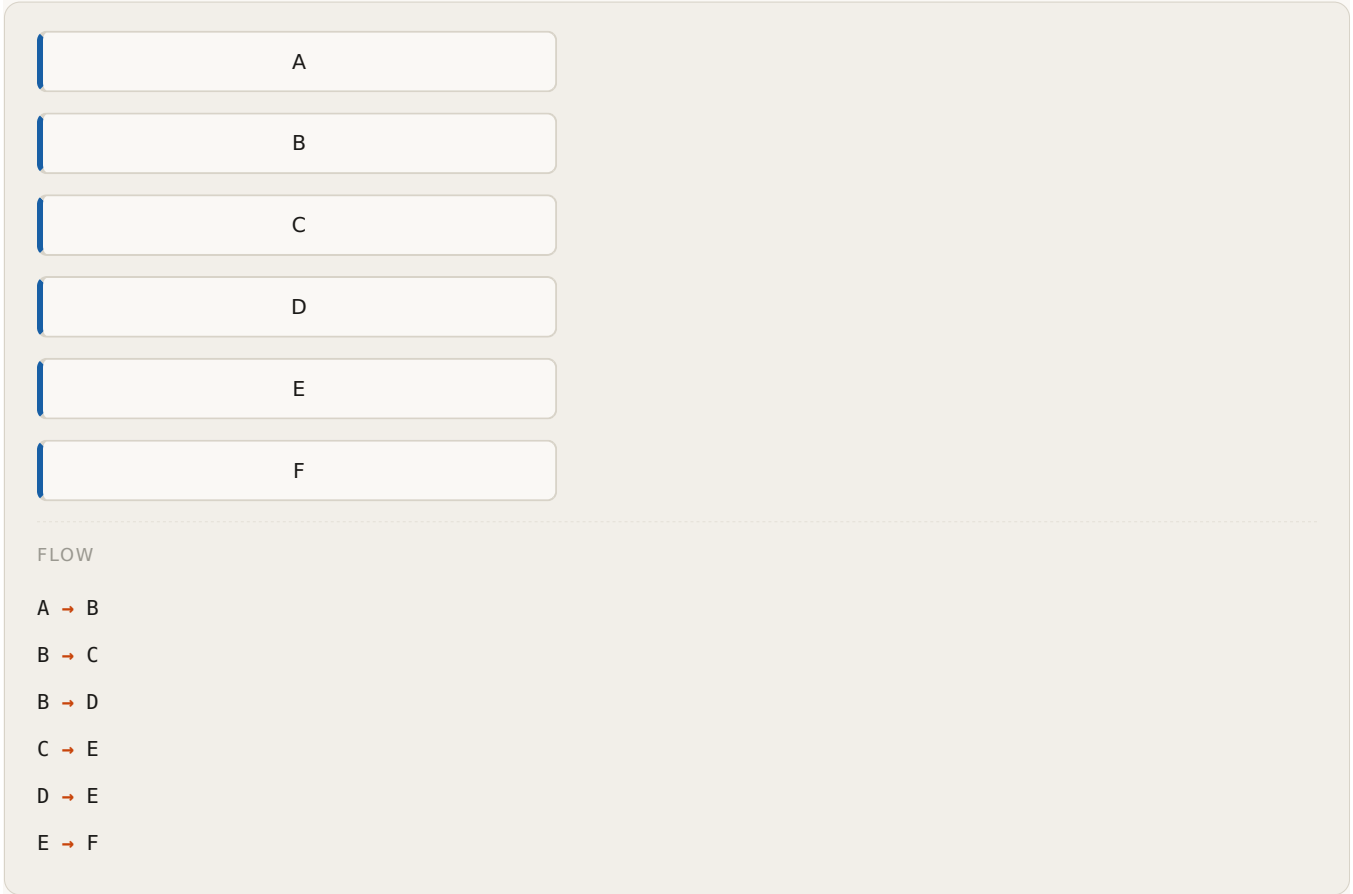
---

3. Word2Vec: Learning Static Embeddings

The Core Idea

Distributional hypothesis: Words appearing in similar contexts have similar meanings.

Word2Vec's solution: Train a neural network to predict context words, then use the learned weights as embeddings.



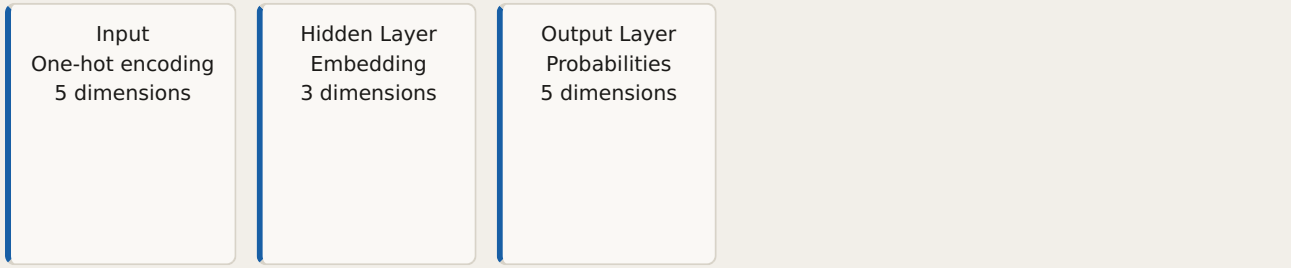
Skip-gram Architecture

Task: Given a center word, predict surrounding words.

Example: "The cat sat on the mat" with window size 2:

```
Center word: "sat"
Context words: "The" (2 before), "cat" (1 before), "on" (1 after), "the" (2 after)

Training pairs generated:
("sat", "The"), ("sat", "cat"), ("sat", "on"), ("sat", "the")
```

The Neural Network



FLOW

Input → Hidden Layer

Hidden Layer → Output Layer

Input → Hidden Layer

Hidden Layer → Output Layer

One-Hot Encoding

```
Vocabulary: ["the", "cat", "sat", "on", "mat"

"the" → [1, 0, 0, 0, 0]
"cat" → [0, 1, 0, 0, 0]
"sat" → [0, 0, 1, 0, 0]
"on"  → [0, 0, 0, 1, 0]
"mat" → [0, 0, 0, 0, 1]
```

The Weight Matrices: W1 and W2

W1 (INPUT → HIDDEN): THE EMBEDDING MATRIX

```
Shape: (Vocabulary size × Embedding dimension) = (5 × 3)

         Emb1  Emb2  Emb3
"the":   0.2   0.5   0.1    ← Row = embedding for "the"
"cat":   0.8   0.3   0.4    ← Row = embedding for "cat"
"sat":   0.1   0.9   0.2
"on":    0.7   0.2   0.5
"mat":   0.4   0.6   0.8
```

Each ROW is a word's embedding!

W2 (HIDDEN → OUTPUT): THE CONTEXT MATRIX

```
Shape: (Embedding dimension × Vocabulary size) = (3 × 5)

        "the"  "cat"  "sat"  "on"  "mat"
Emb1:   0.3    0.1    0.6    0.2   0.4
Emb2:   0.5    0.8    0.2    0.7   0.3
Emb3:   0.1    0.4    0.5    0.3   0.9
```

Each COLUMN represents a word as a context word.

Why Two Matrices?

	W1	W2
Represents	Words as center words	Words as context words
Role	"What am I looking for in my context?"	"What do I offer when I appear in context?"
Shape	Vocab × Embedding	Embedding × Vocab
Used after training	YES (primary embeddings)	Sometimes (or averaged with W1)

They learn together - W1["cat"] and W2[:,"sat"] become similar because "cat" and "sat" co-occur. They push each other to be compatible through training.

Forward Pass: Step by Step

Training example: Input = "cat", Target = "sat"

Step 1: One-hot input

```
input = [0, 1, 0, 0, 0]
```

Step 2: Multiply by W1 (embedding lookup)

```
hidden = W1.T × input

Because input is one-hot, this SELECTS row 1 from W1:

[0.2  0.8  0.1  0.7  0.4]   [0]   [0]
[0.5  0.3  0.9  0.2  0.6] × [1] = [0.3]  ← only row for "cat" survives!
[0.1  0.4  0.2  0.5  0.8]   [0]   [0.4]
                            [0]
                            [0]

hidden = [0.8, 0.3, 0.4]  ← "cat"'s embedding!
```

Why one-hot extracts a row: All zeros except one "1" → only the column corresponding to "cat" survives multiplication. Everything else gets zeroed out!

Step 3: Multiply by W2 to get scores

```
output_scores = W2.T × hidden

[0.3  0.5  0.1]   [0.8]   [0.43]  ← score for "the"
[0.1  0.8  0.4] × [0.3] = [0.48]  ← score for "cat"
[0.6  0.2  0.5]   [0.4]   [0.74]  ← score for "sat" ← HIGHEST!
[0.2  0.7  0.3]           [0.49]  ← score for "on"
[0.4  0.3  0.9]           [0.77]  ← score for "mat"
```

Step 4: Apply softmax

```
Probabilities: [0.17, 0.18, 0.23, 0.18, 0.24]
              "the" "cat" "sat" "on"  "mat"
```

Step 5: Calculate loss

```
Target: "sat" (index 2)
Loss = -log(0.23) = 1.47  ← High loss, model is wrong!
```

Negative Sampling: The Key Optimization

Problem: Softmax over 50,000 words is VERY expensive.

Solution: Instead of predicting ALL words, just distinguish neighbors from random words.

```
Positive: ("cat", "sat") → Label: 1
Negative: ("cat", "airplane") → Label: 0
Negative: ("cat", "computer") → Label: 0
```

Why not use ALL non-neighbors as negatives?

```
Dataset: 1 million words
Non-neighbors per word: ~999,995 words

Total negative examples: BILLIONS → too expensive!

With random sampling:
5-20 negatives per positive → tractable!
```

Training with negative sampling:

```
Positive pair ("cat", "sat"):
  Score = dot(embedding_cat, embedding_sat) = 1.22
  Probability = sigmoid(1.22) = 0.77
  Target = 1
  Loss = -log(0.77) = 0.26
  → Push embeddings CLOSER together

Negative pair ("cat", "airplane"):
  Score = dot(embedding_cat, embedding_airplane) = 0.48
  Probability = sigmoid(0.48) = 0.62
  Target = 0
  Loss = -log(1 - 0.62) = 0.97
  → Push embeddings FARTHER apart
```

Backpropagation in Word2Vec

Gradient descent rule:

```
W_new = W_old - learning_rate × gradient
```

Important: We SUBTRACT the gradient because: - Gradient points toward INCREASING loss (uphill) - We want to DECREASE loss (downhill) - Subtracting a negative gradient = adding to it (the value increases!)

```
gradient = [0.02, -0.03, 0.01]

W1["cat"] = [0.8, 0.3, 0.4] - 0.01 × [0.02, -0.03, 0.01]
          = [0.8, 0.3, 0.4] - [0.0002, -0.0003, 0.0001]
          = [0.7998, 0.3003, 0.3999]

Note: Dimension 2 INCREASED (0.3 → 0.3003)
      because gradient was -0.03 (negative)!
```

What Word2Vec Learns

```
After training on millions of examples:

"cat" → [0.65, 0.48, 0.72]
"dog" → [0.63, 0.50, 0.71]  ← Similar to cat (both pets)!
```

```
"bank" → [0.45, 0.38, 0.51]  ← Compromise between:
                                river context AND
                                financial context

Mathematical relationships:
king - man + woman ≈ queen
```

The "bank" Problem in Word2Vec

```
Training corpus:
"I went to the river bank"      → bank moves toward: river, water
"deposit money at the bank"     → bank moves toward: deposit, money

After training:
"bank" = SINGLE compromise vector

dot(bank, river) = 0.72    ← High
dot(bank, deposit) = 0.68  ← Also high!

Word2Vec CANNOT distinguish which "bank" is meant
in any given sentence!

This is the core limitation of STATIC embeddings.
```

Static vs Contextual Embeddings

	WORD2VEC	TRANSFORMER
"bank" in "river bank"	[0.45, 0.38, 0.51]	[0.91, 0.13, 0.82, ...]
"bank" in "deposit bank"	[0.45, 0.38, 0.51]	[0.15, 0.87, 0.31, ...]
Same vector?	YES (static)	NO (contextual)

---

4. Matrices and Matrix Multiplication

What Is a Matrix?

A grid of numbers arranged in rows and columns:

```
Matrix A (2×3):          Matrix B (3×2):
[1  2  3]                [7   8]
[4  5  6]                [9  10]
                         [11 12]
```

Matrix Multiplication Rules

Rule: A (m×n) × B (n×p) = C (m×p)

The INNER dimensions must match:

```
(2×3) × (3×2) = (2×2) ✓
       ↑   ↑
     These must match!
```

How It Works: Row × Column

```
A (2×3) × B (3×2):

Result[0,0] = Row 0 of A · Col 0 of B
            = 1×7 + 2×9 + 3×11 = 58

Result[0,1] = Row 0 of A · Col 1 of B
            = 1×8 + 2×10 + 3×12 = 64

Result[1,0] = Row 1 of A · Col 0 of B
            = 4×7 + 5×9 + 6×11 = 139

Result[1,1] = Row 1 of A · Col 1 of B
            = 4×8 + 5×10 + 6×12 = 154

Final:
A × B = [58   64 ]
        [139  154]
```

Matrix × Vector (Most Common in ML)

```
W (2×3) × x (3×1):

[1  2  3]   [10]   [1×10 + 2×20 + 3×30]   [140]
[4  5  6] × [20] = [4×10 + 5×20 + 6×30] = [320]
            [30]
```

Intuition: Each row of W creates a weighted combination of x's elements.

Dot Product

Two vectors of same size:

```
a = [2, 3, 4]
b = [5, 6, 7]

dot(a, b) = 2×5 + 3×6 + 4×7 = 56
```

High dot product = vectors point in similar directions = similar words!

---

5. Recurrent Neural Networks (RNNs)

The Problem RNNs Solve

Regular neural networks can't handle sequences:

```
Regular NN:
"The cat sat" → [process] → output
"sat cat The" → [process] → same output!  ← ORDER IGNORED!

RNN:
"The cat sat" → [process with memory] → different output
"sat cat The" → [process with memory] → different output!
```

The RNN Cell: Core Equation

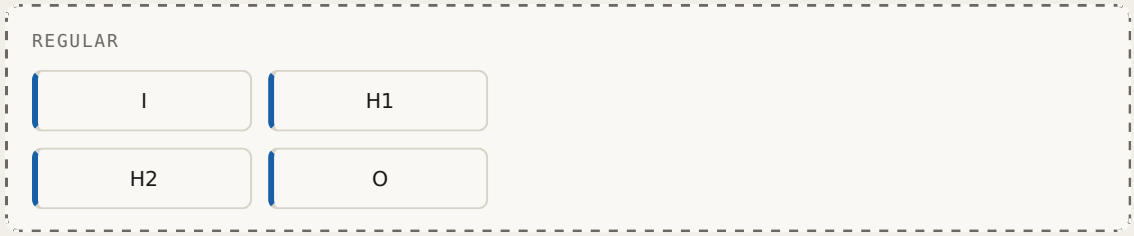
```
h_t = tanh(W_x × x_t + W_h × h_{t-1} + b)
```

Every symbol explained:

SYMBOL	MEANING	DIMENSIONS
h_t	Hidden state at time t (the "memory")	hidden_dim
x_t	Input at time t (current word embedding)	embedding_dim
h_{t-1}	Hidden state from PREVIOUS time step	hidden_dim
W_x	Weight matrix for input (LEARNED)	hidden_dim × embedding_dim
W_h	Weight matrix for hidden state (LEARNED)	hidden_dim × hidden_dim
b	Bias term (LEARNED)	hidden_dim
tanh	Squashes values to [-1, 1]	-

Hidden States vs Hidden Layers

Critical distinction:

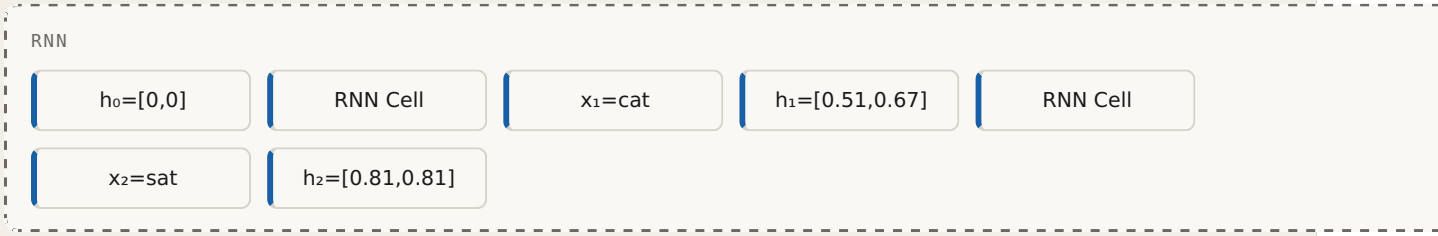


FLOW

I → H1

H1 → H2

H2 → O



FLOW

h₀=[0,0] → RNN Cell

x₁=cat → RNN Cell

RNN Cell → h₁=[0.51,0.67]

h₁=[0.51,0.67] → RNN Cell

x₂=sat → RNN Cell

RNN Cell → h₂=[0.81,0.81]

- **Hidden layers** = layers stacked in SPACE (exist simultaneously)
- **Hidden states** = memory passed through TIME (sequential)

The hidden state h_t is the **OUTPUT** of the RNN cell at each time step - NOT a layer itself!

Same Weights at Every Time Step

Why reuse the same W_x and W_h?

If we used different weights per position:

```
W_x_position_1, W_x_position_2, ..., W_x_position_100

Problems:
1. Enormous parameter count (100× more weights)
2. "cat" at position 1 vs position 50 = completely different!
3. Can't handle variable-length sequences!
```

With shared weights, the model learns ONE general rule:

```
"How to combine current input with previous context"

Works for ANY position:
h₁ = f(x₁, h₀)   ← same f
h₂ = f(x₂, h₁)   ← same f
h₅₀ = f(x₅₀, h₄₉) ← same f
```

Concrete Example: Processing "cat sat"

Setup:

```
embedding_cat = [0.5, 0.3, 0.8]
embedding_sat = [0.2, 0.7, 0.4]
h_0 = [0.0, 0.0]  # Initial hidden state (zeros)

W_x = [[0.3, 0.5, 0.2],
       [0.4, 0.1, 0.6]]

W_h = [[0.8, 0.2],
       [0.3, 0.7]]
```

```
b = [0.1, 0.1]
```

Time Step 1: Processing "cat"

```
x₁ = [0.5, 0.3, 0.8]

W_x × x₁:
Row 1: 0.3×0.5 + 0.5×0.3 + 0.2×0.8 = 0.46
Row 2: 0.4×0.5 + 0.1×0.3 + 0.6×0.8 = 0.71
= [0.46, 0.71]

W_h × h₀:
= [0.0, 0.0]  (zeros × anything = 0)

z₁ = [0.46, 0.71] + [0.0, 0.0] + [0.1, 0.1] = [0.56, 0.81]

h₁ = tanh([0.56, 0.81]) = [0.508, 0.669]
```

Time Step 2: Processing "sat"

```
x₂ = [0.2, 0.7, 0.4]
h₁ = [0.508, 0.669]  ← carries "cat" memory!

W_x × x₂:
Row 1: 0.3×0.2 + 0.5×0.7 + 0.2×0.4 = 0.49
Row 2: 0.4×0.2 + 0.1×0.7 + 0.6×0.4 = 0.39
= [0.49, 0.39]

W_h × h₁:                    ← THIS IS THE KEY!
Row 1: 0.8×0.508 + 0.2×0.669 = 0.540
Row 2: 0.3×0.508 + 0.7×0.669 = 0.620
= [0.540, 0.620]

z₂ = [0.49, 0.39] + [0.540, 0.620] + [0.1, 0.1] = [1.13, 1.11]

h₂ = tanh([1.13, 1.11]) = [0.810, 0.805]
```

h_2 = [0.810, 0.805] contains information about BOTH "cat" AND "sat"!

How Same Weights Produce Different Context Representations

"bank" in different contexts produces different hidden states:

```
"river bank":
h_river = [0.6, 0.3]   ← after processing "river"

h_bank  = tanh(W_x × x_bank + W_h × [0.6, 0.3] + b)
                                    ↑
                              carries "river" info!
= [0.82, 0.21]


"deposit bank":
h_deposit = [0.2, 0.8]  ← after processing "deposit"

h_bank    = tanh(W_x × x_bank + W_h × [0.2, 0.8] + b)
                                     ↑
                               carries "deposit" info!
= [0.19, 0.91]
```

Same W_x, W_h + same x_bank embedding + DIFFERENT h_{t-1} = DIFFERENT output!

The difference comes entirely from h_{t-1} carrying different history.

The Unrolled RNN

FLOW

```
h₀ → RNN Cell
x₁ → RNN Cell
RNN Cell → h₁
h₁ → RNN Cell
x₂ → RNN Cell
RNN Cell → h₂
h₂ → RNN Cell
x₃ → RNN Cell
RNN Cell → h₃
```

Same W_x, W_h used at EVERY cell!

Making Predictions

```
y_t = softmax(W_y × h_t + b_y)

Example: After "cat" (h₁ = [0.508, 0.669])
Scores: [0.555, 0.737, 0.472, 0.720]
         cat    sat    on     mat

Probabilities: [0.23, 0.28, 0.21, 0.27]
Model predicts "sat" is most likely next word!
```

RNN Use Cases

A

B

C

D

B1

B2

B3

B4

C1

C2

C3

D1

D2

D3

FLOW

A → B

A → C

A → D

B → B1

B → B2

B → B3

B → B4

C → C1

C → C2

C → C3

D → D1

D → D2

D → D3

RNN Architectures

```
1. Many-to-One (Sentiment):
   Input:  [I, loved, this, movie]
   Output: [positive]

2. Many-to-Many same length (POS Tagging):
   Input:  [The, cat, sat]
   Output: [DET, NOUN, VERB]

3. Encoder-Decoder / Many-to-Many different length (Translation):
   Input:  [The, cat, sat]
   Output: [Le, chat, assis]
```

W_x vs Word2Vec W1: Important Distinction

A common source of confusion:

	WORD2VEC W1	W_X IN RNN
Purpose	Token ID → embedding vector	Embedding vector → hidden space
Input	Token ID (integer)	Embedding vector
Output	Embedding vector	Part of hidden state computation
Shape	Vocab × Embedding (50,000 × 300)	Hidden × Embedding (256 × 300)
Analogy	Dictionary (look up meaning)	Translator (convert to RNN's language)

They are DIFFERENT matrices appearing at DIFFERENT stages:

```
"cat"
  ↓
Token ID: 2653
  ↓ [Word2Vec W1 / Embedding Matrix]
x_t = [0.65, 0.48, 0.72, ...]   ← 300-dim embedding
  ↓ [W_x: transforms for RNN]
[part of hidden state computation]
  ↓
h_t = [0.82, 0.21, ...]          ← 256-dim hidden state
```

---

6. Vanishing Gradients

Backpropagation Through Time (BPTT)

To train an RNN, we backpropagate gradients through time:

FLOW

h₃ → h₂

h₂ → h₁

h₁ → h₀

At each step, gradient is multiplied by:

```
∂h_t/∂h_{t-1} = diag(tanh'(z_t)) × W_h
```

For T time steps:

```
∂Loss/∂h₁ = ∂Loss/∂h_T × (W_h × tanh')^T
```

The Math of Vanishing

tanh derivative values:

```
tanh'(z) = 1 - tanh²(z)

z = 0.0: tanh' = 1.0  (max)
z = 0.5: tanh' = 0.79
z = 1.0: tanh' = 0.42
z = 2.0: tanh' = 0.07
z = 3.0: tanh' = 0.01
```

Typical value ≈ 0.3 to 0.5

Combined with W_h values (~0.3 to 0.7):

```
Factor per step = tanh' × W_h ≈ 0.4
```

What happens over many steps:

```
Step 50 (output):  gradient = 1.000
Step 45:           gradient = 0.010
Step 40:           gradient = 0.0001
Step 35:           gradient = 0.000001
Step 1:            gradient ≈ 0

gradient at step 1 = 1.0 × (0.4)^50 ≈ 10^{-20} ≈ 0
```

Why Same Weights Make This Worse

With different weights per position (hypothetical):

```
∂Loss/∂h₁ = ∂Loss/∂h_T × W_hT × W_h(T-1) × ... × W_h1
(different matrices, could cancel out sometimes)
```

With SAME weights (actual RNN):

```
∂Loss/∂h₁ = ∂Loss/∂h_T × (W_h)^T
(same matrix raised to power T → guaranteed decay!)
```

The Learning Consequence

```
Word update rule: W_h_new = W_h_old - learning_rate × gradient

If gradient ≈ 0:
W_h_new = W_h_old - 0.01 × 0.000000000001
        ≈ W_h_old  (virtually no change!)

Result: Words far back in sequence contribute NOTHING to learning!
```

Gradient Flow Visualization

```
Sentence: "The quick brown fox jumps over the lazy dog sat"
           ↑                                          ↑
         word 1                                    word 10

Gradient signal strength:
word 10: ████████████████████  (1.000)
word 9:  ████████████████      (0.400)
word 8:  ████████              (0.160)
word 7:  ████                  (0.064)
word 6:  ██                    (0.026)
word 5:  ▌                     (0.010)
word 4:  ▌                     (0.004)
word 3:  |                     (0.002)
word 2:  ·                     (0.001)
word 1:  ·                     (0.0004)

Model CANNOT learn "The ... sat" relationship!
```

---

7. LSTM: Solving Vanishing Gradients

The Key Innovation: Cell State Highway

RNN has ONE path for information and gradients:

```
h₁ → h₂ → h₃ → h₄ → ...
(gradients multiply by W_h × tanh' at every step)
```

LSTM adds a SEPARATE "highway" for long-term memory:

```
c₁ → c₂ → c₃ → c₄ → ...  ← Cell state (highway!)
h₁ → h₂ → h₃ → h₄ → ...  ← Hidden state (as before)
```

LSTM Gates

x_t + h_{t-1}

Forget Gate
f_t = σ(W_f·[h,x] + b_f)
What to forget?

Input Gate
i_t = σ(W_i·[h,x] + b_i)
What new info to add?

Candidate
c̃_t = tanh(W_c·[h,x] + b_c)
New candidate values

Output Gate
o_t = σ(W_o·[h,x] + b_o)
What to output?

Cell State Update
c_t = f_t ⊙ c_{t-1} + i_t ⊙ c̃_t

Hidden State
h_t = o_t ⊙ tanh(c_t)

FLOW

x_t + h_{t-1} → Forget Gate

x_t + h_{t-1} → Input Gate

x_t + h_{t-1} → Candidate

x_t + h_{t-1} → Output Gate

Forget Gate → Cell State Update

Input Gate → Cell State Update

Candidate → Cell State Update

Cell State Update → Hidden State

Output Gate → Hidden State

LSTM Equations

```
# 1. Forget gate: What fraction of old memory to keep?
f_t = sigmoid(W_f × [h_{t-1}, x_t] + b_f)

# 2. Input gate: How much new info to add?
i_t = sigmoid(W_i × [h_{t-1}, x_t] + b_i)

# 3. Candidate: What new info to potentially add?
c̃_t = tanh(W_c × [h_{t-1}, x_t] + b_c)

# 4. Cell state update (THE KEY EQUATION)
c_t = f_t ⊙ c_{t-1} + i_t ⊙ c̃_t
      ↑forget old        ↑add new
```

```
# 5. Output gate: What to expose to next cell?
o_t = sigmoid(W_o × [h_{t-1}, x_t] + b_o)

# 6. Hidden state
h_t = o_t ⊙ tanh(c_t)

Note: ⊙ = element-wise multiplication
      σ = sigmoid (output between 0 and 1)
```

How LSTM Solves Vanishing Gradients

RNN gradient path:

```
∂Loss/∂h₁ ∝ (W_h × tanh')^50 ≈ 0
Fixed, always small factor!
```

LSTM gradient path through cell state:

```
∂c_t/∂c_{t-1} = f_t

∂c_50/∂c_0 = f_50 × f_49 × ... × f_1
```

The crucial difference: f_t is LEARNED and can be close to 1!

```
If f_t ≈ 1 (keep everything):
  gradient = 1 × 1 × 1 × ... × 1 = 1.0  ← No vanishing!

If f_t ≈ 0 (forget everything):
  gradient = 0 × ... = 0  ← Intentional forgetting!

The model LEARNS when to preserve vs forget!
```

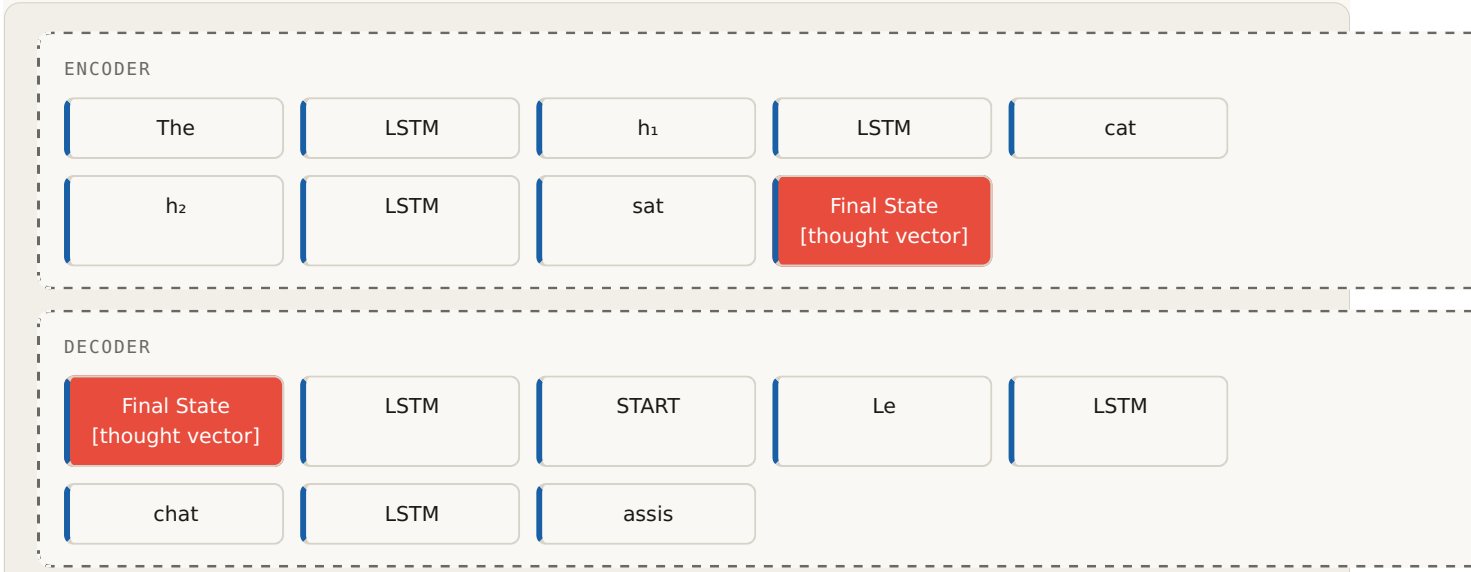
RNN vs LSTM Gradient Flow

```
RNN:
word 10: ████████████████████ (1.000)
word 5:  ▌                    (0.010)
word 1:  ·                    (0.0004)  ← Almost gone!

LSTM (cell state):
word 10: ████████████████████ (1.000)
word 5:  ████████████████▌    (0.850)
word 1:  ████████████▌        (0.650)  ← Still strong!
```

---

8. Encoder-Decoder Architecture

The Concept



FLOW

```
The → LSTM
LSTM → h₁
h₁ → LSTM
cat → LSTM
LSTM → h₂
h₂ → LSTM
sat → LSTM
LSTM → Final State
Final State → LSTM
START → LSTM
LSTM → Le
LSTM → LSTM
Le → LSTM
LSTM → chat
LSTM → LSTM
chat → LSTM
LSTM → assis
```

Step by Step

Task: Translate "cat sat" to French "chat assis"

Encoder Phase:

```
h₀_enc = [0, 0]  # Initialize
```

```
# Process "cat"
h₁_enc = LSTM_enc("cat", h₀_enc) = [0.4, 0.5]

# Process "sat"
h₂_enc = LSTM_enc("sat", h₁_enc) = [0.7, 0.6]

# This is the "thought vector" - compressed meaning of "cat sat"
final_state = [0.7, 0.6]
```

Decoder Phase:

```
h₀_dec = final_state = [0.7, 0.6]  # Seed decoder!

# Generate first word
h₁_dec = LSTM_dec(<START>, h₀_dec)
output_1 = softmax(W_out × h₁_dec) → "chat"

# Generate second word
h₂_dec = LSTM_dec("chat", h₁_dec)
output_2 = softmax(W_out × h₂_dec) → "assis"

# Stop
h₃_dec = LSTM_dec("assis", h₂_dec)
output_3 = softmax(W_out × h₃_dec) → <END>
```

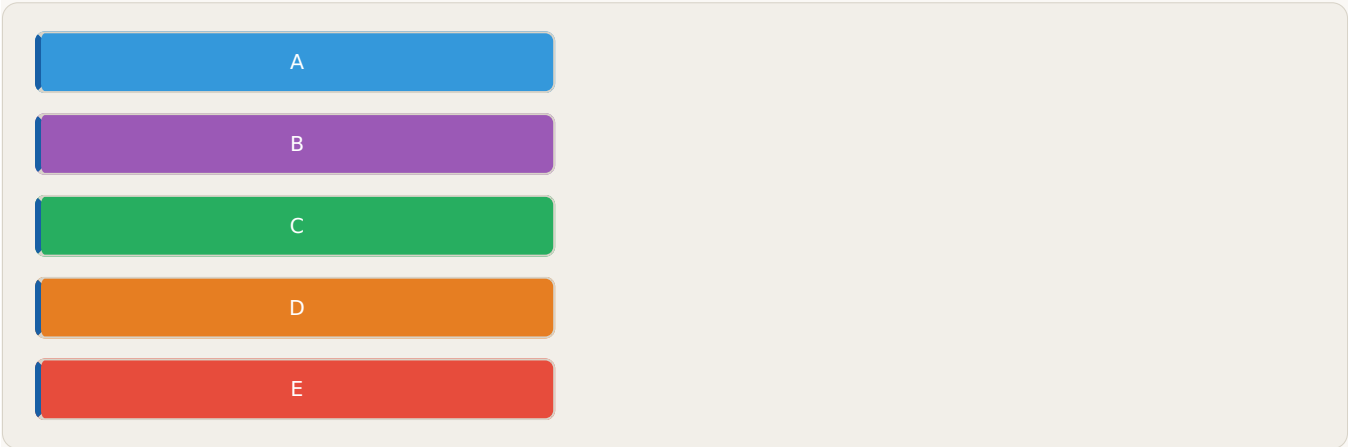
The Bottleneck Problem

```
Long sentence: "The quick brown fox jumps over the lazy dog"
                              ↓
                  final_state = [0.5, 0.8]
                              ↓
                   Everything compressed here!
                   Decoder must reconstruct ENTIRE meaning
                   from just 2 numbers (or 256, or 1024...)
```

This is why attention was invented - let the decoder look back at ALL encoder states, not just the final one!

---

9. From RNNs to Transformers: The Evolution



Architecture Comparison

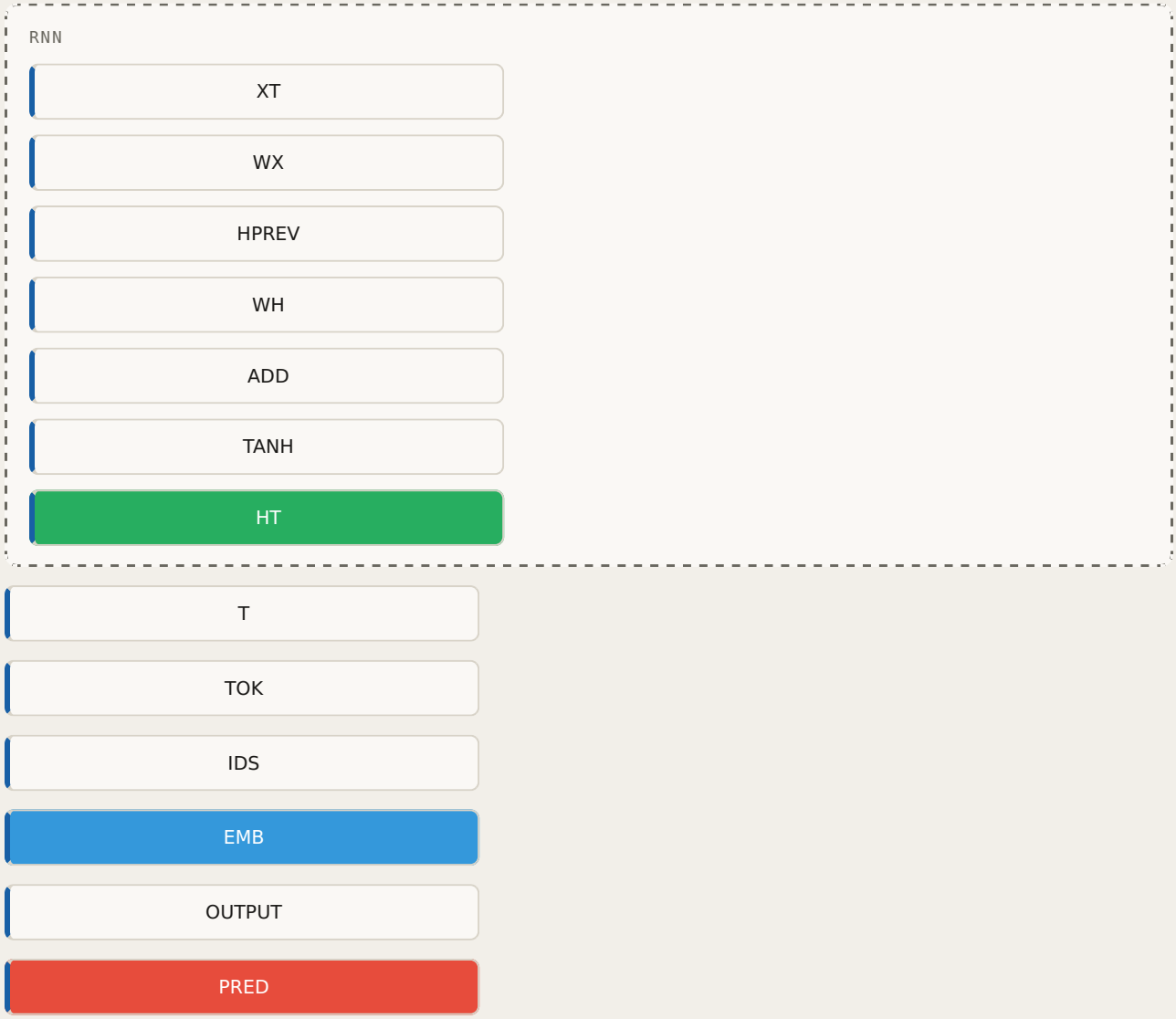
FEATURE	WORD2VEC	RNN	LSTM	TRANSFORMER
Embeddings	Static	Static input	Static input	Contextual
Sequential?	No	Yes	Yes	No (parallel!)
Long-range	N/A	Poor	Better	Excellent
Vanishing gradients	N/A	Severe	Solved	Not applicable
Training speed	Fast	Slow	Slow	Fast (parallel)
Context window	Window	Full sequence	Full sequence	Full sequence
"bank" disambiguation	No	Partial	Better	Yes!

What Each Architecture Adds

```
Word2Vec:
"bank" → [0.45, 0.38, 0.51]  (static, compromise vector)
Knows: bank co-occurs with river AND deposit
Doesn't know: which meaning in THIS sentence

RNN:
"bank" in "river bank" → h_t = [0.82, 0.21]
"bank" in "deposit bank" → h_t = [0.19, 0.91]
Knows: which meaning based on PAST context
Doesn't know: words AFTER "bank"

LSTM:
Same as RNN but can remember LONGER context
Still doesn't see future words

Transformer:
"bank" in "river bank" → [0.91, 0.13, 0.82, ...]
"bank" in "deposit bank" → [0.15, 0.87, 0.31, ...]
Knows: FULL context, both before AND after "bank"
Attends directly to "river" or "deposit" regardless of distance!
```

10. Key Relationships Between Concepts

The Complete Pipeline



FLOW

T → TOK

TOK → IDS

IDS → EMB

EMB → XT

XT → WX

HPREV → WH

WX → ADD

WH → ADD

ADD → TANH

TANH → HT

HT → HPREV

HT → OUTPUT

OUTPUT → PRED

Word2Vec W1 vs Embedding Matrix vs W_x

```
Word2Vec W1:
- Trained separately in Word2Vec
- Rows = word embeddings
- Used AS INPUT to RNN (becomes x_t)
- Shape: (vocab_size × embedding_dim)

Embedding Matrix in transformers:
- Trained end-to-end with the model
- Same structure as W1 but learned jointly
- Shape: (vocab_size × d_model)

W_x in RNN:
- DIFFERENT from embedding matrix!
- Takes embedding vector (x_t)
- Transforms it to hidden state space
- Shape: (hidden_dim × embedding_dim)
```

Training Signals Summary

```
Word2Vec:
Signal: Predict context words
Updates: W1 (embeddings) + W2 (context matrix)
Result: Words with similar context → similar embeddings

RNN:
Signal: Predict next word (or other task)
Updates: W_x, W_h, W_y (all weights)
Result: Sequential patterns captured in hidden states

LSTM:
Signal: Same as RNN
Updates: W_f, W_i, W_c, W_o (four gate matrices)
Result: Long-range patterns captured via cell state
```

Quick Reference: Key Formulas

Word2Vec

```
Positive pair loss:  L = -log(sigmoid(e_w · e_c))
Negative pair loss:  L = -log(sigmoid(-e_w · e_r))
Weight update:       W_new = W_old - lr × gradient
```

RNN

```
Hidden state:   h_t = tanh(W_x·x_t + W_h·h_{t-1} + b)
Output:         y_t = softmax(W_y·h_t + b_y)
Loss:           L = -log(P(target | h_t))
```

LSTM

```
Forget gate:    f_t = σ(W_f·[h_{t-1},x_t] + b_f)
Input gate:     i_t = σ(W_i·[h_{t-1},x_t] + b_i)
Candidate:      č_t = tanh(W_c·[h_{t-1},x_t] + b_c)
Cell update:    c_t = f_t⊙c_{t-1} + i_t⊙č_t
```

```
Output gate:    o_t = σ(W_o·[h_{t-1},x_t] + b_o)
Hidden state:   h_t = o_t⊙tanh(c_t)
```

Common Confusions Clarified

1. "bank" in Word2Vec

> *Word2Vec gives "bank" ONE vector that's a compromise between all its contexts. It moves closer to BOTH "river" AND "deposit" during training. It cannot distinguish which meaning is intended in a specific sentence.*

2. Hidden States vs Hidden Layers

> *Hidden layers = layers stacked in space in a regular NN. Hidden states = memory passed through time in RNN. They are completely different concepts!*

3. W_x vs Word2Vec W1

> *W1 creates embeddings from token IDs. W_x transforms those embeddings into hidden state space. They appear at different stages and have different purposes.*

4. Why Same Weights?

> *RNNs reuse weights for every time step to: (1) handle variable-length sequences, (2) generalize across positions, (3) reduce parameter count. The downside: the same repeated multiplication causes vanishing gradients.*

5. Vanishing Gradients

> *Gradients vanish because: same small factor (W_h × tanh' ≈ 0.4) is multiplied 50+ times. (0.4)^50 ≈ 0. Early words receive essentially zero gradient signal and cannot influence learning.*

6. LSTM Solution

> *LSTM adds a cell state highway where gradient flows through multiplication by f_t (forget gate) instead of W_h × tanh'. Since f_t is LEARNED and can be close to 1, the gradient can flow unchanged over many steps.*

This document covers the journey from static word embeddings through RNNs to the doorstep of Transformers. The next chapter covers Attention Mechanisms, Self-Attention, Q/K/V matrices, and the full Transformer architecture.

PART II

The Conceptual Thread

Notes from Hands-On Large Language Models. The lineage: Word2Vec's projection idea → RNN inherits embeddings → LSTM adds a highway → Transformer generalises both.

> Short, high-signal notes. If you read nothing else, read the 'Conceptual Thread' diagram — it's the single best map of the field.

Running notes from reading sessions. Added progressively.

1. The Three-Part Structure of a Transformer LLM

A Transformer LLM has three main components:

```
Tokenizer → Stack of Transformer Blocks → LM Head → next token
```

- **Tokenizer**: Converts raw text into token IDs using a vocabulary table (e.g. 50,000 tokens)
- **Transformer Blocks**: N stacked layers that progressively build contextual understanding
- **LM Head**: Final projection that converts embeddings into next-token probabilities

2. The LM Head — Converting Embeddings to Token Probabilities

After all Transformer blocks finish, each token has a rich contextual embedding vector (e.g. 768 dims). The LM head converts this into a probability over every word in the vocabulary.

The Process

```
768-dim vector → multiply by W_lm (768 × 50,000) → 50,000 logits → softmax → probabilities → pick token ID → tokenizer lookup → actual word
```

Concrete Example (tiny model: 4 dims, 5 tokens)

```
Embedding vector: x = [0.8, 0.2, 0.6, 0.1]

W_lm matrix (4 × 5):
         cat   dog   sat   the   mat
dim 0  [ 0.9,  0.1,  0.3,  0.2,  0.4 ]
dim 1  [ 0.1,  0.8,  0.2,  0.1,  0.3 ]
dim 2  [ 0.3,  0.2,  0.7,  0.1,  0.5 ]
dim 3  [ 0.2,  0.1,  0.4,  0.9,  0.1 ]

score("sat") = (0.8×0.3) + (0.2×0.2) + (0.6×0.7) + (0.1×0.4) = 0.74

After softmax:
  cat: 28%,  dog: 8%,  sat: 24%,  the: 8%,  mat: 16%
```

What "Learned" Means

The numbers inside W_lm start random and get adjusted through backpropagation over billions of training examples. After training they are fixed. During inference the matrix is just applied as-is.

LM Head vs Multi-Head Attention

These are completely different things that happen to share the word "head": - **Multi-head attention**: Inside each Transformer block. Multiple parallel attention streams, each specializing in a different relationship type (subject-verb, location, etc.) - **LM head**: After all Transformer blocks. A single weight matrix that maps from embedding space to vocabulary space.

---

3. One Complete Transformer Layer

```
Input x (+ Positional Encoding on first layer only)
    ↓
Self-Attention (QKV lookup, multi-head)
    ↓
+ x  (residual addition)
    ↓
LayerNorm
    ↓
FFN (expand → ReLU → compress)
    ↓
+ x  (residual addition)
    ↓
LayerNorm
    ↓
Output vector (same shape as input)
```

In pseudocode:

```
def transformer_layer(x):
    z      = layer_norm(x + multi_head_attention(x))
    output = layer_norm(z + FFN(z))
    return output
```

PE is only added once — at the very beginning before Layer 1. All subsequent layers receive only the output vector from the previous layer.

---

4. Residual Connections — Why They Exist and How They Work

The Problem: Vanishing Gradients

In a deep network, backpropagation passes the gradient backward through every layer using the chain rule — each layer multiplies the gradient by its own derivative. If those derivatives are small (e.g. 0.1), the gradient shrinks exponentially:

```
Loss gradient:    -8.0
After Layer 2:    -8.0 × 0.1 = -0.8
After Layer 1:    -0.8 × 0.1 = -0.08   ← tiny with just 2 layers

With 12 layers:   -8.0 × 0.1^12 = -0.000000008   ← completely vanished
```

Early layers learn essentially nothing because their gradient signal disappears.

The Solution: Residual Connection

Change each layer's formula from:

```
output = f(x)           ← no residual
```

to:

```
output = x + f(x)       ← with residual
```

Why This Fixes the Gradient — Concrete Example

Setup:

```
Layer 1: f(x) = 2x  (derivative = 2)
Layer 2: g(x) = 3x  (derivative = 3)
Input x = 1.0,  Target = 10.0
Loss = (prediction - target)²
```

Forward pass WITH residual:

```
After Layer 1:  1.0 + 2(1.0) = 3.0
After Layer 2:  3.0 + 3(3.0) = 12.0
Loss = (12.0 - 10.0)² = 4.0
```

Backward pass WITH residual:

Each layer now differentiates as `d(x + f(x))/dx = 1 + d(f(x))/dx`

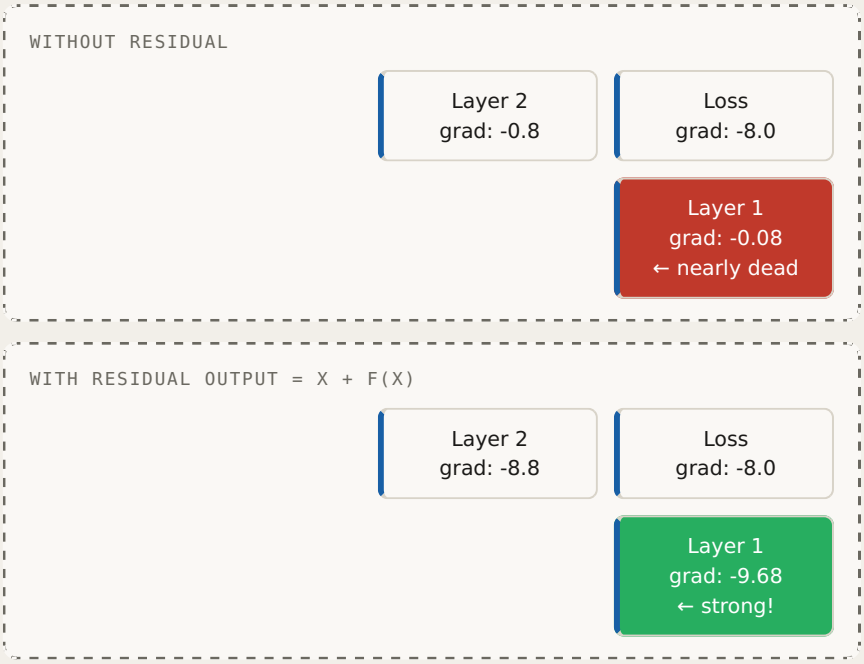
```
Loss gradient = 2 × (12.0 - 10.0) = 4.0

Layer 2 derivative = 1 + 3 = 4      (1 from residual + 3 from g(x)=3x)
Gradient at Layer 2 input = 4.0 × 4 = 16.0

Layer 1 derivative = 1 + 2 = 3      (1 from residual + 2 from f(x)=2x)
Gradient at Layer 1 input = 16.0 × 3 = 48.0
```

The Key: The "1" Is Not Magic

The `1` in `1 + d(f(x))/dx` is not artificially added. It's the natural derivative of `x` with respect to itself — mathematically it must always be 1. So no matter how small `d(f(x))/dx` gets, the total gradient through any layer is always at least 1.

Comparison with small derivatives (0.1):

```
                    WITHOUT residual    WITH residual
Layer derivative:         0.1              1 + 0.1 = 1.1
Loss gradient:           -8.0                -8.0
After Layer 2:      -8.0 × 0.1 = -0.8   -8.0 × 1.1 = -8.8
After Layer 1:      -0.8 × 0.1 = -0.08  -8.8 × 1.1 = -9.68
```



FLOW

Loss *[× 0.1]* → Layer 2

Layer 2 *[× 0.1]* → Layer 1

Loss *[× 1.1*
(1 + 0.1)] → Layer 2

Layer 2 *[× 1.1*
(1 + 0.1)] → Layer 1

The correction signal reaches every layer cleanly, no matter how deep the network.

Two Gradient Paths

The residual creates two paths for the gradient to travel backward:

```
Gradient from loss
    ├── Path 1: through f(x) transformation  → × small derivative
    └── Path 2: through + x shortcut         → × 1.0 always
         ↓
Both paths add together at the input
```

The Plain English Version

Training a deep network is like fixing a mistake that happened 12 steps ago in an assembly line. Without residuals, your correction message passes through all 12 workers, each garbling it slightly — by step 1, it's a whisper. With residuals, there's also a direct intercom to every step. The correction arrives intact regardless of depth.

---

5. Exploding Gradients — The Opposite Problem

Residual connections guarantee gradients don't vanish, but could they grow too large? Yes — this is called **exploding gradients**. Three mechanisms keep this in check:

1. Small learning rate:

```
new_weight = old_weight - (learning_rate × gradient)
learning_rate ≈ 0.0001

Even gradient of -9.68 becomes:
update = 0.0001 × 9.68 = 0.000968   ← tiny nudge
```

2. Layer Normalization: Rescales outputs to mean=0, std=1 after each residual addition, preventing values from growing uncontrollably as they pass through layers.

3. Gradient clipping: During training, if the gradient exceeds a threshold (e.g. 1.0), it gets scaled down. Standard safety net in LLM training.

Residual connections + LayerNorm + learning rate + gradient clipping work together as a system: - Residuals → prevent vanishing - LayerNorm + clipping + learning rate → prevent exploding

---

More notes to be added as reading continues.

---

6. The Conceptual Thread: Word2Vec → RNN → Transformer

This section documents the beautiful conceptual lineage that connects all major NLP architectures. Each one inherited the best ideas from its predecessor and solved what was broken.

The Core Problem Each Architecture Solved

```
Word2Vec:     How do we represent meaning as numbers?
RNN:          How do we handle sequences and word order?
LSTM:         How do we remember important things over long sequences?
Transformer:  How do we capture context without sequential processing?
```

Each architecture stood on the shoulders of the previous one.

Chapter 1: Word2Vec — Projection Reveals Meaning

Word2Vec has two weight matrices: W1 and W2.

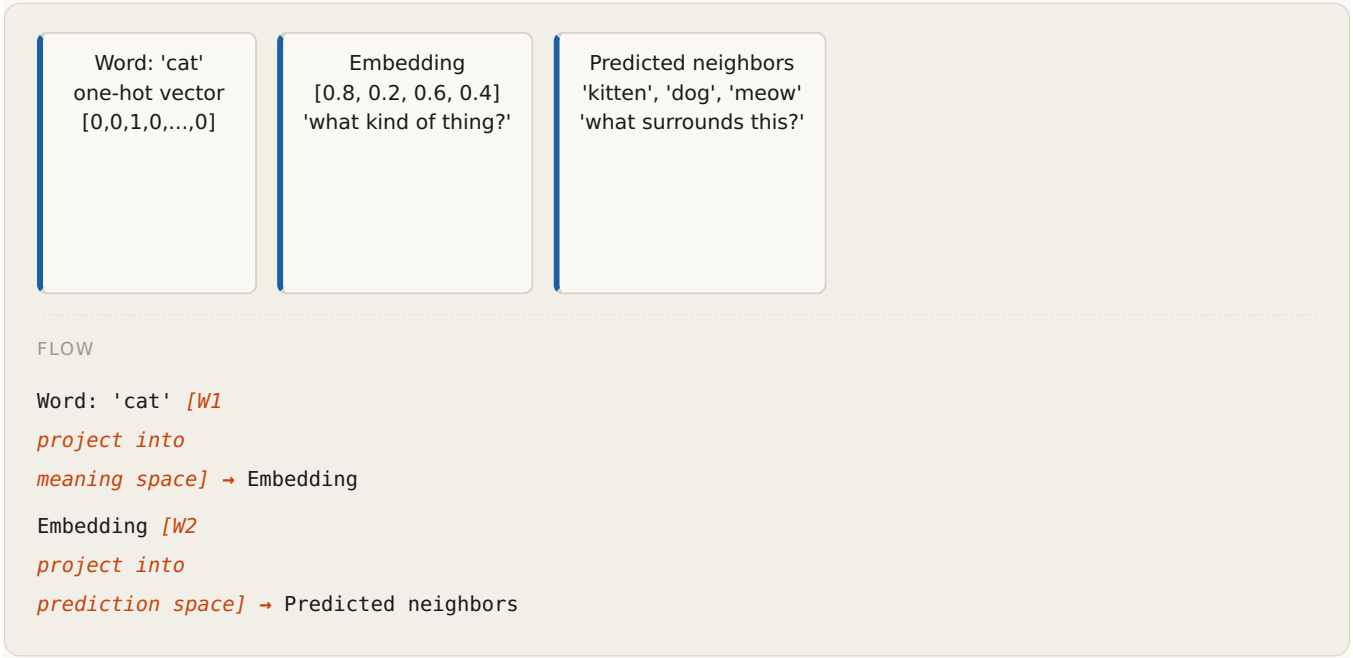
```
Input word (one-hot vector)
    ↓
W1  ← projects word into embedding space
    ↓
```

```
Hidden layer (the embedding)
    ↓
W2  ← projects embedding into prediction space
    ↓
Predicted context words
```

The key insight: **W1 is a projection matrix**. It takes a word and asks "what kind of thing is this word?" The answer is the embedding — a point in semantic space where similar words cluster together.

W2 then asks: "given what kind of thing this is, what words tend to appear around it?"

Training adjusts both matrices until the embeddings genuinely capture meaning. The famous result: `king - man + woman = queen` falls out naturally from the geometry.



The fundamental idea: a learned projection matrix transforms a vector into a space where a specific question becomes answerable.

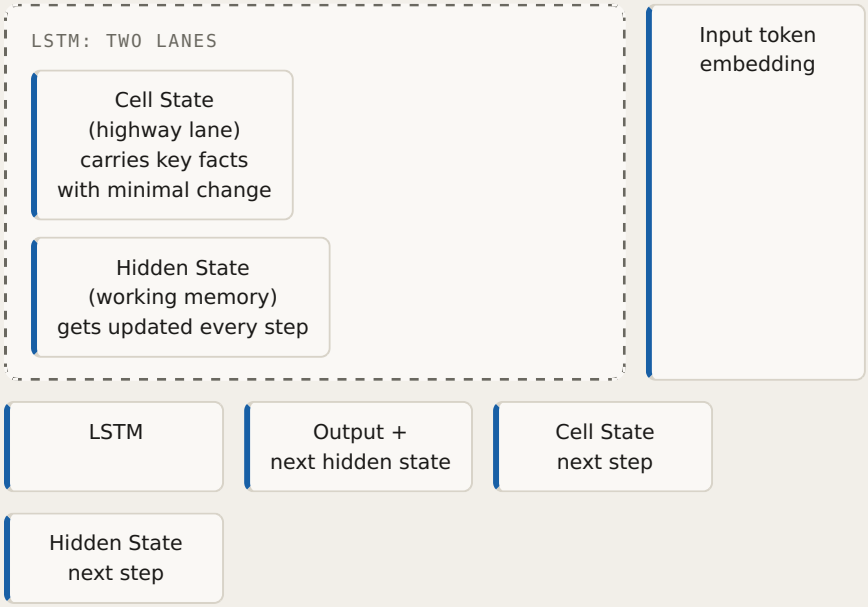
Chapter 2: RNN — Embeddings Meet Sequence

RNNs inherited Word2Vec embeddings directly. The embedding of each word became the input to the RNN at each time step.

The problem: that single memory vector h has to carry everything. By token 50, what happened at token 1 is nearly gone. This is the vanishing gradient problem across time.

Chapter 3: LSTM Cell State — The Fast Lane for Important Information

LSTM introduced the **cell state** — a dedicated lane that runs parallel to the normal hidden state, specifically designed to carry important information forward with minimal transformation.



FLOW

```
Input token → LSTM
LSTM → Output +
Cell State [flows through
with small changes
like a highlighter] → Cell State
Hidden State [heavily transformed
every step] → Hidden State
```

The key insight: important information needs a protected path that doesn't get overwritten by every new token. The cell state is that path.

The remaining problem: still sequential. Token 2 waits for token 1. Cannot parallelise. Slow to train.

Chapter 4: Transformer Attention — The W1/W2 Insight Reborn

Here is the beautiful conceptual inheritance. Remember W1 in Word2Vec: *a learned projection that transforms a vector to answer a specific question?*

Attention uses **three** such projections simultaneously:

```
W_Q  "What am I looking for?"        → produces Query vector
W_K  "What do I offer to others?"    → produces Key vector
W_V  "What information do I carry?"  → produces Value vector
```

This is W1 and W2's insight, but now applied three ways at once to enable a rich lookup mechanism.


WORD2VEC (THE ANCESTOR)
Input word
Embedding
Prediction
TRANSFORMER ATTENTION (THE DESCENDANT)
Token embedding
Query
Key
Value
Attention scores
Context-enriched vector
W2V
ATT
FLOW
Input word [W1 'what kind of thing?'] → Embedding
Embedding [W2 'what neighbors?'] → Prediction
Token embedding [W_Q 'what am I looking for?'] → Query
Token embedding [W_K 'what do I offer?'] → Key
Token embedding [W_V 'what info do I carry?'] → Value
Query [Q·K relevance scoring] → Attention
Attention [softmax × V] → Context-enriched

Key → Attention

Value → Context-enriched

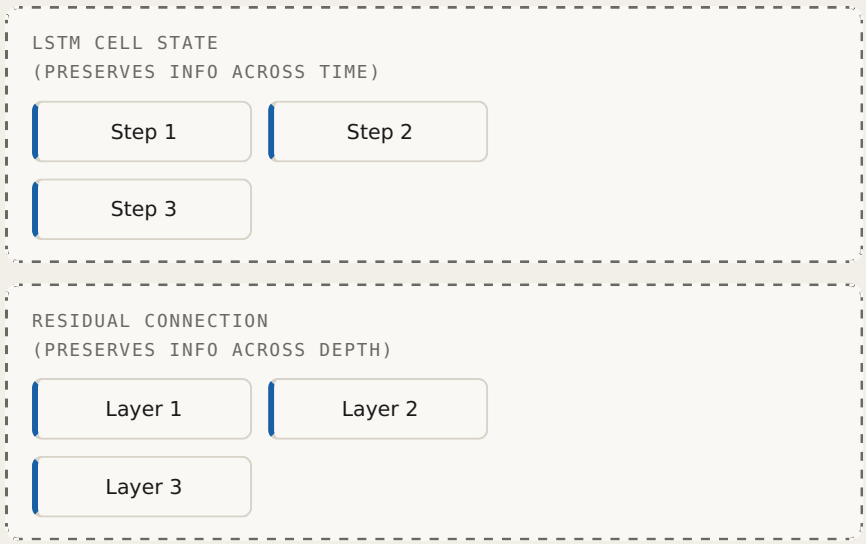
W2V *[same core idea: learned projection reveals relationships]* → ATT

The conceptual leap: Word2Vec used projections to find semantic neighbors in a fixed vocabulary. Attention uses projections to find contextually relevant tokens in the current sequence — dynamically, based on content.

Chapter 5: Residual Connections — The Cell State Idea for Depth

The cell state solved "how do we preserve important information across time (RNN steps)?"

Residual connections solve the exact same problem but across **depth** (Transformer layers):



FLOW

Step 1 *[cell state bypasses transformation]* → Step 2

Step 2 *[cell state bypasses transformation]* → Step 3

Layer 1 *[x + f(x) bypasses transformation]* → Layer 2

Layer 2 *[x + f(x) bypasses transformation]* → Layer 3

Both solve the same problem with the same solution: - Information needs to travel a long distance (across steps or across layers) - Without a bypass, transformations progressively destroy the signal - A direct lane (cell state / residual) lets the original signal arrive intact

---

The Complete Lineage

Word2Vec
Learned projections W1,W2
capture semantic meaning
words as vectors in space

RNN
Inherits Word2Vec embeddings
adds sequential processing
memory vector carries context

LSTM
Inherits RNN structure
adds cell state bypass
protects important info across time

Transformer
Inherits embedding idea
W1,W2 insight → W_Q, W_K, W_V
cell state idea → residual connections
parallelises everything

FLOW

Word2Vec *[embeddings become RNN inputs]* → RNN

RNN *[sequential memory not enough]* → LSTM

LSTM *[cell state insight generalised to depth]* → Transformer

Word2Vec *[projection insight generalised to attention]* → Transformer

---

The Single Unifying Idea Across All Architectures

Every architecture in NLP history is trying to answer one question:

> ***Given a sequence of words, what is the richest possible representation of meaning that lets us predict what comes next?***

- Word2Vec: meaning lives in geometric space
- RNN: meaning must accumulate over sequence
- LSTM: important meaning needs a protected lane
- Transformer: meaning comes from direct comparison of all tokens simultaneously, through learned projections

Each solved what the previous couldn't. The Transformer didn't discard its predecessors — it generalised their best ideas and parallelised everything.

PART III

Transformer Architecture

The full decoder-only transformer, piece by piece. Positional encoding, self-attention, multi-head attention, residuals, LayerNorm, feed-forward, causal masking, generation.

> Treat this as the theoretical companion to Part IV. Every named component in this part shows up as code — often `nn.something` — in the nanochat build.

From Positional Encoding to Text Generation

Table of Contents

1. The Problem Transformers Solve

What Was Wrong Before

Before transformers, the two main approaches each had fundamental limitations:

```
Word2Vec (2013): Static embeddings
──────────────────────────────────
"bank" always = [0.45, 0.38, 0.51]
ONE vector regardless of context.
Training corpus had both "river bank" and "deposit bank",
so the vector is a compromise between both meanings.
Cannot tell which meaning is intended in any specific sentence!

RNN (2014-2017): Sequential context
────────────────────────────────────
"bank" gets context from the PAST only.
"The river bank was..." → h_bank contains river info ✓
BUT "bank was flooded"  → can't see "flooded" when processing "bank"!
Also: vanishing gradients make long-range learning impossible.
Sequential processing is slow — can't parallelize.

Transformer (2017): Full parallel context
─────────────────────────────────────────
"bank" sees ALL words simultaneously — past AND future.
No vanishing gradients.
All tokens processed IN PARALLEL (very fast on GPUs!).
```

The Single Most Important Intuition

RNN = Telephone game. Information degrades at every step.

```
"The" → whispers to → "river" → whispers to → "bank" → whispers to → "was"

By the time gradient reaches "The" from "flooded", it has passed through
many tanh and W_h multiplications. It is nearly zero. The model
cannot learn that "The river..." and "...bank" are related!
```

Transformer = Direct conversation. Every word talks directly to every other word.

```
"The"     ←──────────────────────────────────→ "bank"
"river"   ←──────────────────────────────────→ "bank"
"was"     ←──────────────────────────────────→ "bank"
"flooded" ←──────────────────────────────────→ "bank"

"bank" directly asks EACH word:
"How relevant are you to understanding ME?"
Distance does not matter. No information degrades.
```

---

2. The Big Picture: Full Pipeline

TRANSFORMER LAYER ×12 (BERT) OR ×96 (GPT-3)

Multi-Head Self-Attention
Every token attends to every token
8 heads in parallel
Context gathered here

Add Residual + LayerNorm
x = LayerNorm(x + attention_output)

Feed-Forward Network
Each token processed independently
W1 → ReLU → W2
Context digested here

Add Residual + LayerNorm
x = LayerNorm(x + ff_output)

Raw Text
'The river bank was flooded'

B

C

D

E

Final Contextual Embeddings
5 vectors × 768 dimensions
Now fully context-aware!
'bank' = geographic meaning

Prediction Head
Last token's vector
→ Linear layer → 50,257 scores
→ Softmax → probabilities

Next Token: 'flooded'
(highest probability)

FLOW

Raw Text → B

B → C

C → D

D → E

Multi-Head Self-Attention → Add Residual + LayerNorm

Add Residual + LayerNorm → Feed-Forward Network

Feed-Forward Network → Add Residual + LayerNorm

```
E → Final Contextual Embeddings

Final Contextual Embeddings → Prediction Head

Prediction Head → Next Token: 'flooded'
```

Critical observation: The input shape (5 tokens × 768 dim) and output shape (5 tokens × 768 dim) are **identical**. Each layer refines the representations in place — it never changes their size.

---

3. Positional Encoding

Why We Need It

Self-attention compares every token against every other token simultaneously. This means it has **no built-in sense of order** — it treats the input as a set, not a sequence.

```
Without positional encoding:

"cat bit dog"  → attention sees {cat, bit, dog}
"dog bit cat"  → attention sees {dog, bit, cat}

Both produce IDENTICAL attention patterns!
The model cannot tell which word came first.
"cat" as subject vs "cat" as object — indistinguishable!
```

Why Not Just Use 1, 2, 3?

The naive approach of adding position numbers fails in several ways:

```
Option A — add raw position number:
"bank" at position 2:   [0.25, 0.47] + 2   = [2.25, 2.47]
"bank" at position 100: [0.25, 0.47] + 100 = [100.25, 100.47]
Problem: Values explode at large positions. Training becomes unstable.

Option B — normalize to [0, 1]:
pos 2 in a 10-word sentence  → 2/10  = 0.20
pos 2 in a 100-word sentence → 2/100 = 0.02
Problem: Same absolute position gets different values
         depending on sentence length. Confusing!

Option C — learned embeddings (one per position):
Learn a vector for position 0, 1, 2, ...
Problem: Cannot generalize to sequences longer
         than the longest training example!
```

The Sine/Cosine Solution

Each position gets a **vector of the same size as d_model**, computed from sine and cosine waves at different frequencies:

```
PE(pos, 2i)   = sin( pos / 10000^(2i / d_model) )
PE(pos, 2i+1) = cos( pos / 10000^(2i / d_model) )

where:
  pos     = position in the sentence (0, 1, 2, ...)
  i       = which pair of dimensions (0, 1, 2, ...)
  d_model = total embedding size (768 for BERT, etc.)
  10000   = large constant controlling the frequency range
```

Properties: - Values always stay between ∝'1 and 1 → no explosion - Every position gets a unique fingerprint - Works for any sequence length - Relative distances are consistent (see below)

Positional Encoding Vector Size

```
PE vector size  =  d_model  (e.g. 768 for BERT)

NOT vocabulary size! These are completely different things:

  Vocabulary size (50,257) = number of ROWS in embedding matrix
                           = how many different tokens exist

  d_model (768)            = number of DIMENSIONS per token vector

We add PE to an embedding element-by-element:
  embedding  = [0.25, 0.47, 0.38, ..., 0.51]   768 values
  PE_pos3    = [0.14, 0.99, 0.28, ..., -0.96]  768 values
  x_input    = embedding + PE_pos3              768 values ✓

We CANNOT add a 768-dim vector to a 50,257-dim vector.
PE size must match embedding size.
```

How PE Gets Added: Element-wise Addition

```
Sentence: "deposit at the bank"
Positions:    0       1   2   3

"bank" is at position 3.

Step 1 — embedding lookup (pure word meaning, no position):
  bank_embedding = [0.25, 0.47, 0.38, 0.51]

Step 2 — compute PE for position 3 (pure position, no meaning):
  PE_pos3 = [sin(3/1), cos(3/1), sin(3/100), cos(3/100)]
          = [0.14,    0.99,     0.03,        1.00     ]
          (4-dim example; in practice 768 dim)

Step 3 — add element-by-element:
  x_final = [0.25+0.14, 0.47+0.99, 0.38+0.03, 0.51+1.00]
          = [0.39,      1.46,      0.41,       1.51     ]

Now x_final carries BOTH word meaning AND position information!
```

Same Word, Different Position → Different Starting Vector

```
"bank" in "deposit at the bank"         (position 3)
  embedding  = [0.25, 0.47, 0.38, 0.51]  ← same always
  PE_pos3    = [0.14, 0.99, 0.03, 1.00]
  x_input    = [0.39, 1.46, 0.41, 1.51]  ← unique to pos 3

"bank" in "the old river bank nearby"   (position 3 vs position 4)
If "bank" appears at position 4:
  embedding  = [0.25, 0.47, 0.38, 0.51]  ← same always
  PE_pos4    = [∝'0.76, 0.65, 0.04, 1.00]  ← different PE!
  x_input    = [∝'0.51, 1.12, 0.42, 1.51]  ← different!

The model can distinguish "bank" appearing early vs late in the sentence.
```

Why Both Sine AND Cosine?

```
If we only used sin:
  sin(0)     = 0.000
  sin(3.14)  = 0.000  ← positions 0 and Ї€ are identical!
```

```
  Ambiguous!

With paired sin+cos:
  position 0:   (sin=0.000, cos=1.000)   ← unique point on unit circle
  position Ï€:   (sin=0.000, cos=∝'1.000)  ← different point!

No two positions produce the same (sin, cos) pair.
Think of a clock: every hour is uniquely identified
by its (sin, cos) coordinate!
```

Multiple Frequencies: The Clock Analogy

Different dimension pairs use different frequencies, like clock hands:

```
d_model = 8 → 4 (sin, cos) pairs, indexed i = 0, 1, 2, 3

i=0: denominator = 10000^(0/8) = 1
     Changes FAST → completes a full cycle in ~6 positions
     Good for: distinguishing immediately adjacent words

i=1: denominator = 10000^(2/8) ≈ 18
     Changes MEDIUM → one cycle per ~100 positions
     Good for: medium-range relationships

i=2: denominator = 10000^(4/8) = 100
     Changes SLOW → one cycle per ~600 positions
     Good for: sentence-level structure

i=3: denominator = 10000^(6/8) ≈ 562
     Changes VERY SLOW → distinguishes global position

Together: like reading a clock with 4 hands.
Any position in a long sequence can be uniquely identified!
```

```
Positional fingerprints (4-dim example):

Pos:     dim0      dim1     dim2    dim3
  0   [  0.000,   1.000,  0.000,  1.000 ]
  1   [  0.841,   0.540,  0.010,  0.999 ]
  2   [  0.909,  -0.416,  0.020,  0.999 ]
  3   [  0.141,  -0.990,  0.030,  0.999 ]
  4   [ -0.757,  -0.654,  0.040,  0.999 ]

Every row is unique. Values always within [∝'1, 1].
Early dims (i=0) change fast → nearby position differences.
Late dims (i=3) change slow  → global position.
```

How the Model Learns Relative Position

The model can learn "position 5 is 3 steps after position 2" because any fixed offset `k` corresponds to a **fixed rotation** applied to the (sin, cos) coordinates. This rotation is the same regardless of which two positions you compare. The attention mechanism can learn these rotations through training and generalize to any relative distance.

---

4. Self-Attention: The Core Idea

The YouTube Search Analogy

Self-attention is best understood as a **database query**:

QUERY
'how to make pasta'
What you are looking for

KEY: Video 1
'Pasta recipe - easy'
What Video 1 advertises

KEY: Video 2
'Car repair tutorial'
What Video 2 advertises

KEY: Video 3
'Italian pasta from scratch'
What Video 3 advertises

VALUE: Video 1
Actual pasta recipe content

VALUE: Video 2
Car repair content — irrelevant

VALUE: Video 3
Authentic pasta tutorial content

OUTPUT
Weighted mix of content
from high-matching videos

FLOW

QUERY *[match = HIGH]* → KEY: Video 1

QUERY *[match = LOW]* → KEY: Video 2

QUERY *[match = HIGH]* → KEY: Video 3

KEY: Video 1 → VALUE: Video 1

KEY: Video 2 → VALUE: Video 2

KEY: Video 3 → VALUE: Video 3

VALUE: Video 1 *[weight: 0.50]* → OUTPUT

VALUE: Video 2 *[weight: 0.02]* → OUTPUT

VALUE: Video 3 *[weight: 0.48]* → OUTPUT

Now apply this to the word "sat" attending to its context in **"The cat sat"**:

```
"sat" creates a QUERY:
  Q_sat = "I am a past-tense verb. I need my subject — who performed the action?"

Every word creates a KEY (advertising what it is):
  K_The  = "I am a determiner — not very informative for subjects"
  K_cat  = "I am an animate noun — I could be a subject!"
  K_sat  = "I am a past-tense verb"

"sat" compares its QUERY against all KEYS:
  score(sat → The)  = dot(Q_sat, K_The)  = 0.341  ← low
```

```
  score(sat → cat)  = dot(Q_sat, K_cat)  = 0.919  ← HIGH!
  score(sat → sat)  = dot(Q_sat, K_sat)  = 0.517  ← medium (self)

Every word creates a VALUE (actual semantic content to share):
  V_cat = [0.44, 0.54, 0.70, 0.86]
  "Here is my content: animate noun, potential subject, small animal..."

"sat" collects a WEIGHTED MIX of values:
  new_sat = 0.55 × V_cat + 0.25 × V_sat + 0.20 × V_The
          = [0.33, 0.42, 0.57, 0.70]

"sat" now contains 55% of "cat"'s information!
The subject-verb relationship is captured in a single forward pass.
```

---

5. Q, K, V: Three Learned Transformations

What Each One Does

Every token embedding x is projected into **three separate vector spaces** using three learned weight matrices:

```
Q_t = W_Q × x_t      QUERY  — "What am I looking for?"
K_t = W_K × x_t      KEY    — "What do I offer to others?"
V_t = W_V × x_t      VALUE  — "What content do I actually share?"

W_Q, W_K, W_V are all learned during training.
Each is shape (d_model × d_model), e.g. (768 × 768).
```

Why Three Matrices Instead of One?

Why separate Q and K?

```
If Q = K (same matrix for both):
  "sat" generates: "here is my search query"
  "cat" generates: "here is what I need from context"  ← WRONG!

The ASKING role and the ANSWERING role are different.
Q is specialized to generate good search queries.
K is specialized to generate good advertisements.
Separate matrices let each specialize independently.
```

Why separate K and V?

```
K (Key): optimized for MATCHING
  "Am I the kind of token you are searching for?"
  Like a book's INDEX or search tags.
  Optimized to answer: "should you look at me?"

V (Value): optimized for CONTENT
  "Here is the actual information I carry."
  Like a book's actual TEXT.
  Optimized to answer: "what useful info can I give you?"

Why keep them separate?
The best way to SIGNAL relevance differs from
the best way to DELIVER content!

K_cat might encode: "I am animate, I am a noun, I could be a subject"
                    → signals for matching with verb queries

V_cat might encode: "I am a small pet animal that moves independently"
                    → actual semantic content to transfer
```

```
Separate matrices allow each to be independently optimized!
```

Connection to Word2Vec

```
Word2Vec had two perspectives per word:
  W1 (center word)  ≈ Query  — "what am I looking for in context?"
  W2 (context word) ≈ Key    — "what do I offer when near other words?"

Transformers add a THIRD perspective:
  Q = asking for information                    (like W1)
  K = advertising what you have                 (like W2)
  V = the actual content you share              ← NEW, not in Word2Vec!

The V matrix separates "relevance signaling" from "content delivery."
This is why transformers learn richer representations than Word2Vec.
```

Where Does Attention STRENGTH Come From?

A frequent confusion: **V is the content, not the strength.**

```
Strength of attention = dot(Q, K) → scaled → softmax → attention weight
Content retrieved     = the V vectors themselves

Example for "sat" attending to "cat":

  attention_weight(sat → cat) = 0.55     ← THIS is the strength!
  V_cat = [0.44, 0.54, 0.70, 0.86]       ← THIS is the content!

  Contribution = 0.55 × [0.44, 0.54, 0.70, 0.86]
                  ↑              ↑
              strength        content

V does not control how much attention is paid.
V controls what information flows when attention IS paid.
```

---

6. Self-Attention: Step-by-Step with Numbers

Setup

```
Sentence: "cat sat"  (after embedding lookup + positional encoding)
d_model = 4          (normally 768+, reduced for clarity)

Input embeddings:
  embedding_cat = [0.5, 0.6, 0.7, 0.8]
  embedding_sat = [0.2, 0.3, 0.4, 0.5]

Learned weight matrices (4×4 each):
  W_Q = identity matrix (simplified; normally random and learned)

  W_K = [[0.8, 0.1, 0.0, 0.0],
         [0.1, 0.9, 0.0, 0.0],
         [0.0, 0.0, 0.7, 0.2],
         [0.0, 0.0, 0.2, 0.8]]

  W_V = [[0.5, 0.2, 0.1, 0.0],
         [0.2, 0.6, 0.0, 0.1],
         [0.1, 0.0, 0.7, 0.2],
         [0.0, 0.1, 0.2, 0.8]]
```

Step 1: Create Q, K, V for Every Token

```
Q_cat = W_Q × embedding_cat = [0.5, 0.6, 0.7, 0.8]  (identity → same)
Q_sat = W_Q × embedding_sat = [0.2, 0.3, 0.4, 0.5]

K_cat = W_K × embedding_cat:
  Row 0: 0.8×0.5 + 0.1×0.6 = 0.40 + 0.06 = 0.46
  Row 1: 0.1×0.5 + 0.9×0.6 = 0.05 + 0.54 = 0.59
  Row 2: 0.7×0.7 + 0.2×0.8 = 0.49 + 0.16 = 0.65
  Row 3: 0.2×0.7 + 0.8×0.8 = 0.14 + 0.64 = 0.78
  K_cat = [0.46, 0.59, 0.65, 0.78]

K_sat = W_K × embedding_sat:
  Row 0: 0.8×0.2 + 0.1×0.3 = 0.16 + 0.03 = 0.19
  Row 1: 0.1×0.2 + 0.9×0.3 = 0.02 + 0.27 = 0.29
  Row 2: 0.7×0.4 + 0.2×0.5 = 0.28 + 0.10 = 0.38
  Row 3: 0.2×0.4 + 0.8×0.5 = 0.08 + 0.40 = 0.48
  K_sat = [0.19, 0.29, 0.38, 0.48]

V_cat = W_V × embedding_cat = [0.44, 0.54, 0.70, 0.86]
V_sat = W_V × embedding_sat = [0.20, 0.27, 0.40, 0.51]
```

Step 2: Compute Attention Scores

Each token's Query is dot-producted against every Key:

```
"sat" asking "how relevant is each word to ME?":

  score(sat → cat) = dot(Q_sat, K_cat)
                   = 0.2×0.46 + 0.3×0.59 + 0.4×0.65 + 0.5×0.78
                   = 0.092 + 0.177 + 0.260 + 0.390
                   = 0.919  ← HIGH — sat should attend strongly to cat

  score(sat → sat) = dot(Q_sat, K_sat)
                   = 0.2×0.19 + 0.3×0.29 + 0.4×0.38 + 0.5×0.48
                   = 0.038 + 0.087 + 0.152 + 0.240
                   = 0.517  ← medium — attend to self somewhat

"cat" asking "how relevant is each word to ME?":

  score(cat → cat) = dot(Q_cat, K_cat) = 1.663  ← high self-attention
  score(cat → sat) = dot(Q_cat, K_sat) = 0.919
```

Full score matrix:

```
               K_cat   K_sat
  Q_cat:       1.663   0.919    (cat attends mostly to itself)
  Q_sat:       0.919   0.517    (sat attends mostly to cat!)
```

Step 3: Scale the Scores

```
WHY SCALE?
With d_model = 768 dimensions, dot products can become very large.
Large scores → softmax output approaches a one-hot distribution
             → almost all attention collapses onto a single token.
We want smoother attention distributions.

Scale factor = sqrt(d_k) = sqrt(4) = 2.0

Scaled scores for "sat":
  sat → cat: 0.919 / 2.0 = 0.460
  sat → sat: 0.517 / 2.0 = 0.259
```

Step 4: Softmax → Attention Weights

```
For "sat" (after scaling):
  scores = [0.460, 0.259]  (cat, sat)

  e^0.460 = 1.584
  e^0.259 = 1.296
  sum     = 2.880

  attention_weights = [1.584/2.880, 1.296/2.880]
                    = [0.550,       0.450]
                       ↑ cat         ↑ sat

  "sat" pays 55% attention to "cat"
  "sat" pays 45% attention to itself
  All weights sum to 1.0 ✓

For "cat":
  attention_weights = [0.592, 0.408]  (cat=59%, sat=41%)
```

Step 5: Weighted Sum of Values

Information actually flows between tokens here:

```
new_sat = 0.550 × V_cat + 0.450 × V_sat

  V_cat = [0.44, 0.54, 0.70, 0.86]
  V_sat = [0.20, 0.27, 0.40, 0.51]

  Dim 1: 0.550×0.44 + 0.450×0.20 = 0.242 + 0.090 = 0.332
  Dim 2: 0.550×0.54 + 0.450×0.27 = 0.297 + 0.122 = 0.419
  Dim 3: 0.550×0.70 + 0.450×0.40 = 0.385 + 0.180 = 0.565
  Dim 4: 0.550×0.86 + 0.450×0.51 = 0.473 + 0.230 = 0.703

  new_sat = [0.332, 0.419, 0.565, 0.703]

Compare:
  original embedding_sat = [0.200, 0.300, 0.400, 0.500]
  after self-attention    = [0.332, 0.419, 0.565, 0.703]

"sat" now encodes 55% of "cat"'s information!
The subject-verb relationship is captured.
```

The Complete Formula

```
Attention(Q, K, V)  =  softmax( Q × Káµ€ / √d_k ) × V

Step by step:
  1.  Q × Káµ€           → raw score matrix (seq_len × seq_len)
  2.  ÷ √d_k           → scale to prevent collapse
  3.  softmax(row-wise) → attention weight matrix (sums to 1 per row)
  4.  × V              → weighted combination of values

This is the most important equation in modern AI.
```

---

7. Multi-Head Attention

Why Multiple Heads?

One attention pass captures one type of relationship. Natural language has many simultaneous relationship types:

```
Sentence: "The cat sat on the mat"

Relationship 1 — Subject/verb (syntax):
  "sat" heavily attends to "cat"  (cat is the agent of sat)

Relationship 2 — Verb/location (syntax):
  "sat" heavily attends to "mat"  (mat is where the action happened)

Relationship 3 — Determiner/noun (grammar):
  "The" attends to "cat"
  "the" attends to "mat"

Relationship 4 — Positional proximity:
  Each word attends to its immediate neighbors

One attention head can only learn one dominant pattern.
Multiple heads learn different patterns in parallel.
```

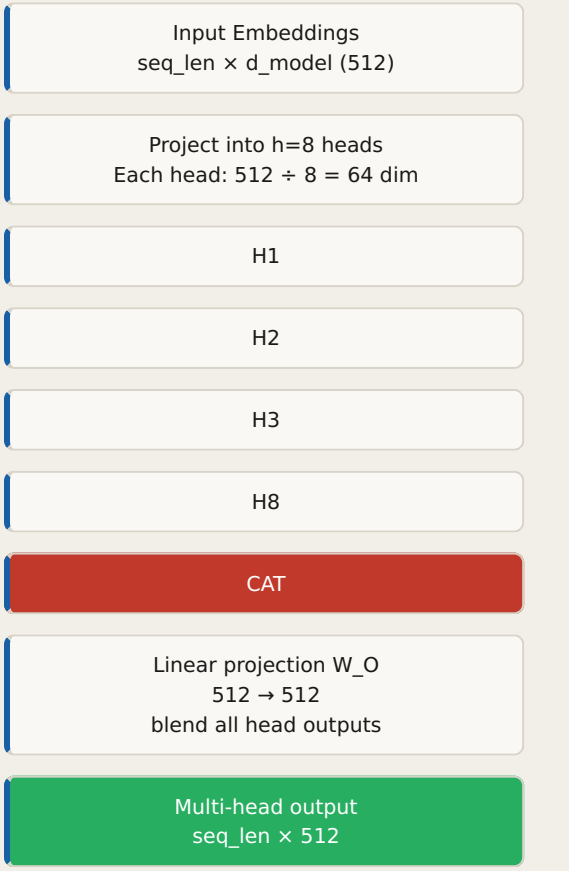
Architecture

```
HEAD 1
W_Q1, W_K1, W_V1
(SYNTAX — SUBJECT/VERB?)
```

```
HEAD 2
W_Q2, W_K2, W_V2
(SEMANTICS — MEANING?)
```

```
HEAD 3
W_Q3, W_K3, W_V3
(POSITION — PROXIMITY?)
```

```
HEAD 8
W_Q8, W_K8, W_V8
(COREFERENCE — 'IT'?)
```



FLOW

```
Input Embeddings → Project into h=8 heads

Project into h=8 heads → H1

Project into h=8 heads → H2

Project into h=8 heads → H3

Project into h=8 heads → H8

H1 → CAT
```

```
H2 → CAT

H3 → CAT

H8 → CAT

CAT → Linear projection W_O

Linear projection W_O → Multi-head output
```

Dimensions

```
d_model = 512    total embedding size
n_heads = 8      number of parallel attention heads
d_head  = 512/8  = 64   dimension per head

Each head gets its own W_Q, W_K, W_V matrices (each 64×512).
Each head computes attention fully independently.
All head outputs are concatenated: 8 × 64 = 512 dim.
Final W_O matrix (512×512) blends all head information.

Why split into smaller heads rather than one large attention?
  Each head only "sees" a 64-dim projection of the embedding.
  Different projections highlight different aspects of meaning.
  Multiple specialized views → richer combined representation.
```

---

8. Residual Connections

The Problem Without Them

In a 12-layer network without residuals, each layer fully overwrites the previous output:

```
WITHOUT residuals:

input → Layer 1 → Layer 2 → ... → Layer 12
         output     output           output
         replaces   replaces         replaces

Problems:
1. If Layer 3 learns a bad transformation, all later layers inherit it.
2. Gradient must flow THROUGH every layer during backprop:
   Layer 12 → ×(attention weights) → Layer 11 → × → ... → Layer 1
   Each multiplication can shrink the gradient.
   By Layer 1 the gradient is nearly zero → vanishing again!
```

The Solution: Add, Don't Replace

```
WITH residuals:

input x ──────────────────────────────────────── ADD → output
          └──→ multi_head_attention(x) ──────────── ADD

output = x + attention(x)

Each layer computes a CORRECTION (delta), not a replacement.
The original information is always preserved.
```

Concrete Example

```
"bank" enters Layer 3:
  bank_input = [0.61, 0.38, 0.55, 0.32]

Attention computes a small contextual correction:
  attention_output = [∝'0.05, +0.12, +0.08, ∝'0.03]

WITHOUT residual:
  bank_after = [∝'0.05, 0.12, 0.08, ∝'0.03]
  Original meaning completely gone!

WITH residual:
  bank_after = bank_input + attention_output
             = [0.61+(∝'0.05), 0.38+0.12, 0.55+0.08, 0.32+(∝'0.03)]
             = [0.56,          0.50,       0.63,       0.29]

  Original information preserved.
  Small contextual correction applied.
  Layer 3 has chipped away a little, not started over.
```

The Sculptor Analogy

```
Start:      rough stone block         ← initial embedding
Layer 1:    + small correction        ← basic syntactic pattern
Layer 2:    + small correction        ← grammatical role
Layer 3:    + small correction        ← semantic context emerging
...
Layer 12:   + small correction        ← full disambiguation

Final result: detailed sculpture      ← fully contextual representation

No layer discards the stone and starts fresh.
Each layer refines what came before.
```

Residuals Fix Gradient Flow Too

```
WITHOUT residuals:
  ∂Loss/∂Layer1 = ∂Loss/∂Layer12 × ∂L12/∂L11 × ... × ∂L2/∂L1
                                   ↑ each term < 1 → product vanishes!

WITH residuals:
  ∂output/∂input = 1 + ∂(correction)/∂input
                   ↑
          This constant "1" ensures gradient is at least 1!
          Direct bypass from output to input at every layer.
          Same idea as LSTM's cell state gradient highway.
```

---

9. Layer Normalization

The Problem It Solves

After the residual addition, the vector values can have wildly different scales:

```
bank_input     = [ 0.61,  0.38,  0.55,  0.32]
+ attention    = [ 2.41, ∝'1.87,  3.22,  0.05]
= result       = [ 3.02, ∝'1.49,  3.77,  0.37]

Dim 0 is +3.77, Dim 1 is ∝'1.49.
The Q, K, V weight matrices in the NEXT layer will have
```

```
drastically different magnitudes multiplied through them.
Training becomes unstable. Convergence suffers.
```

LayerNorm: Step by Step

Step 1 — Calculate the mean of the vector:

```
values = [3.02, ∝'1.49, 3.77, 0.37]

mean = (3.02 + (∝'1.49) + 3.77 + 0.37) / 4
     = 6.67 / 4
     = 1.668
```

Step 2 — Calculate standard deviation:

```
Deviations from mean:
   3.02 ∝' 1.668 =  1.352
  ∝'1.49 ∝' 1.668 = ∝'3.158
   3.77 ∝' 1.668 =  2.102
   0.37 ∝' 1.668 = ∝'1.298

Squared deviations: [1.828, 9.973, 4.418, 1.685]

Variance = (1.828 + 9.973 + 4.418 + 1.685) / 4 = 4.476
Std dev  = √4.476 = 2.116
```

Step 3 — Normalize:

```
normalized = (values ∝' mean) / std

  Dim 0: ( 3.02 ∝' 1.668) / 2.116 =  0.639
  Dim 1: (∝'1.49 ∝' 1.668) / 2.116 = ∝'1.492
  Dim 2: ( 3.77 ∝' 1.668) / 2.116 =  0.993
  Dim 3: ( 0.37 ∝' 1.668) / 2.116 = ∝'0.613

  normalized = [0.639, ∝'1.492, 0.993, ∝'0.613]
  Mean ≈ 0 ✓     Std ≈ 1 ✓
```

Step 4 — Scale and shift with learned parameters Î³ and Î²:

```
Î³ (gamma, scale) = [1.2, 0.8, 1.5, 0.9]    ← learned during training
Î² (beta, shift)  = [0.1, ∝'0.2, 0.0, 0.3]   ← learned during training

output = Î³ × normalized + Î²

  Dim 0: 1.2 × 0.639  + 0.1   =  0.867
  Dim 1: 0.8 × (∝'1.492) ∝' 0.2 = ∝'1.394
  Dim 2: 1.5 × 0.993  + 0.0   =  1.490
  Dim 3: 0.9 × (∝'0.613) + 0.3 = ∝'0.252

  final = [0.867, ∝'1.394, 1.490, ∝'0.252]

Why Î³ and Î²?
Pure normalization always forces mean=0, std=1.
But the model might genuinely need mean=2, std=3 in some dimensions!
Î³ and Î² let the model "undo" normalization where needed,
while still getting the stability benefit during the forward pass.
```

LayerNorm vs BatchNorm

```
BatchNorm (image CNNs):
  Normalizes ACROSS the batch dimension.
  "Compare this pixel's values to the same pixel in other images."
```

```
  Bad for text: batch sizes vary, sequences have different lengths.

LayerNorm (transformers):
  Normalizes WITHIN each token's own vector.
  "Compare this token's dimensions to its own other dimensions."
  Works identically regardless of batch size or sequence length.
  Applied once per token, per layer.
```

---

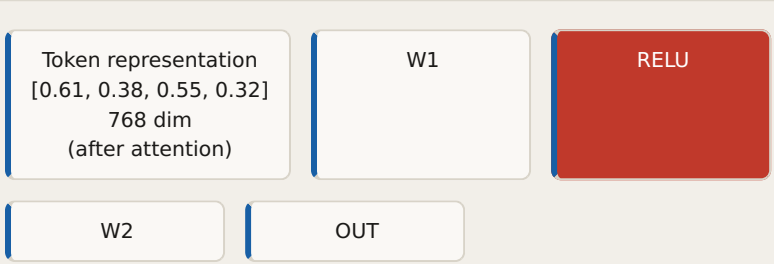
10. Feed-Forward Network

Role in the Transformer

Self-attention and the feed-forward network are complementary operations:

```
Self-attention = GATHERING context
  "I am 'bank'. I have collected info from 'river' and 'flooded'.
   My representation now includes their signals."

Feed-forward = PROCESSING that context
  "Now I will reason about what I have gathered:
   river + flooded + bank = geographical flood event, not a financial bank."
```

Architecture: Expand → Activate → Compress



FLOW

Token representation → W1

W1 → RELU

RELU → W2

W2 → OUT

Step by Step with Numbers

```
Input (bank after attention): x = [0.61, 0.38, 0.55, 0.32]
d_ff = 16  (normally 3072, i.e. 4 × d_model)

── Step 1: W1 expands to larger space (4 → 16 dim) ──

hidden = W1 @ x + b1

  Dim 0:  0.3×0.61 + 0.5×0.38 + 0.1×0.55 + 0.2×0.32 = 0.492
  Dim 1:  0.8×0.61 + 0.1×0.38 + 0.4×0.55 + 0.3×0.32 = 0.842
  Dim 2:  0.2×0.61 + 0.6×0.38 + 0.5×0.55 + 0.1×0.32 = 0.657
  Dim 3:  0.1×0.61 + 0.3×0.38 + 0.8×0.55 + 0.4×0.32 = 0.743
  ...
  Dim 10: (some weight combination)                  = ∝'0.231
  Dim 11: (some weight combination)                  =  0.156
  Dim 12: (some weight combination)                  = ∝'0.445
```

```
  hidden_raw = [0.492, 0.842, 0.657, 0.743, ..., ∝'0.231, 0.156, ∝'0.445, ...]

── Step 2: ReLU removes negatives ──

  ReLU(x) = max(0, x)

  hidden = [0.492, 0.842, 0.657, 0.743, ..., 0.0,   0.156,  0.0, ...]
                                              ↑               ↑
                                          zeroed!          zeroed!

  Interpretation:
    Dim 0:  "Is this geographic?"        → 0.492  YES, active ✓
    Dim 1:  "Is there water involved?"   → 0.842  YES, active ✓
    Dim 2:  "Is this a noun?"            → 0.657  YES, active ✓
    Dim 10: "Is this financial context?" → 0.0    NO,  zeroed ✗
    Dim 12: "Is this about money?"       → 0.0    NO,  zeroed ✗

── Step 3: W2 compresses back (16 → 4 dim) ──

  output = W2 @ hidden + b2
  output = [0.71, 0.29, 0.63, 0.48]

  W2 reads which feature detectors fired and combines
  them into the final 4-dimensional output.
```

Why Expand THEN Compress?

```
4 → 16 → 4

The expanded space (16 dim) is a large "reasoning workspace":
  W1 runs 16 different feature detectors simultaneously.
  ReLU keeps only the ones that fire positively.
  W2 combines the fired detectors into the output.

Without ReLU:
  W2 @ (W1 @ x) = (W2 @ W1) @ x = W_single @ x
  Two matrix multiplications collapse into one!
  No additional expressive power at all.

With ReLU (non-linearity):
  The zeroing creates a non-linear "feature selection" step.
  W1 and W2 genuinely do different things.
  Complex patterns emerge that a single matrix cannot learn.

In practice (768 → 3072 → 768):
  3072 feature detectors run simultaneously per token.
  This is where the model stores much of its factual knowledge.
  Research shows: removing FFN layers hurts factual recall more than
  removing attention layers!
```

Per-Token Independence

```
Self-attention: tokens COMMUNICATE with each other
  "cat" and "sat" exchange signals simultaneously.

Feed-forward: each token REASONS independently
  "cat"'s FFN processes cat's post-attention state.
  "sat"'s FFN processes sat's post-attention state.
  No cross-token communication here.

The two mechanisms are complementary:
  Attention → collect from neighbors
  FFN       → think about what you collected
```

---

11. The Complete Transformer Layer

MULTI-HEAD SELF-ATTENTION
ALL TOKENS ATTEND TO ALL TOKENS
USING H=8 PARALLEL HEADS

Head 1
Q₁K₁áµ€V₁

Head 2
Q₂K₂áµ€V₂

... Head 8
Q₈K₈áµ€V₈

FEED-FORWARD NETWORK
PER TOKEN, INDEPENDENTLY

W1
768→3072
Expand

ReLU
Feature
selection

W2
3072→768
Compress

Input x
seq_len × d_model
(e.g. 5 tokens × 768 dim)

MHA

ADD residual
x + attention(x)
Original info preserved

LayerNorm
mean=0, std=1
Stabilize values

FFN

ADD residual
x + ff(x)
Original info preserved

LayerNorm
Stabilize again

Output
Seq_len × d_model
(same shape as input!)

FLOW

Input x → MHA

MHA → ADD residual

Input x → ADD residual

ADD residual → LayerNorm

LayerNorm → FFN

W1 → ReLU

ReLU → W2

FFN → ADD residual

LayerNorm → ADD residual

ADD residual → LayerNorm

LayerNorm → Output

In pseudocode:

```
def transformer_layer(x):
    # ── Block 1: Self-Attention ──
    attn_out = multi_head_attention(x)    # gather context from all tokens
    x = layer_norm(x + attn_out)          # residual + stabilize

    # ── Block 2: Feed-Forward ──
    ff_out = W2 @ relu(W1 @ x + b1) + b2 # process each token independently
    x = layer_norm(x + ff_out)            # residual + stabilize

    return x   # same shape as input!
```

This block repeats N times — stacking is what makes transformers powerful:

```
Model         Layers  Heads  d_model  Parameters
──────────    ──────  ─────  ───────  ──────────
BERT-base       12      12     768      110M
GPT-2 small     12      12     768      117M
GPT-2 XL        48      25    1600     1.5B
GPT-3           96      96   12288     175B
```

---

12. How Representations Evolve Through Layers

Tracing "bank" Through All Layers

```
Sentence: "The river bank was flooded"

After embedding + PE (Layer 0 input):
  bank = [0.25, 0.47, 0.38, 0.51]
  Static meaning + positional fingerprint.
  Identical to "bank" in "deposit at the bank" at same position!
  Zero context.

After Layer 1-2 (surface patterns):
```

```
  bank = [0.38, 0.52, 0.41, 0.48]
  Slightly influenced by nearby tokens.
  Learns: which words are adjacent, basic co-occurrence.

After Layer 3-5 (syntax):
  bank = [0.61, 0.38, 0.55, 0.32]
  Subject-verb structure learned.
  "river" is a modifier of "bank".
  "was" indicates past tense applied to bank.

After Layer 6-9 (semantics):
  bank = [0.78, 0.21, 0.69, 0.18]
  Word sense disambiguation occurring.
  "river + flooded" context → geographic interpretation confirmed.
  Financial interpretation suppressed.

After Layer 10-12 (reasoning):
  bank = [0.91, 0.13, 0.82, 0.11]
  Full contextual meaning.
  Inference: "bank that was flooded by the river".
  Ready for prediction.
```

What Is Stored Permanently vs Computed Fresh

```
PERMANENTLY IN MEMORY (trained weights, always loaded):

  1. Embedding Matrix   (50,257 × 768)
       One row per vocabulary token. Always static.
       bank → [0.25, 0.47, 0.38, ...]

  2. Positional Encoding Table  (max_seq_len × 768)
       Computed by formula, fixed. Added once before Layer 1.

  3. Per Layer × 12:
       W_Q, W_K, W_V  (for each of 8 heads)
       W_O            output projection
       W1, W2         feed-forward weights
       Î³, Î²           layer norm parameters


COMPUTED FRESH FOR EVERY NEW SENTENCE (temporary):

  4. Q, K, V vectors for each token
       bank_Q = W_Q × bank_input   ← recomputed every inference pass!
       bank_K = W_K × bank_input
       bank_V = W_V × bank_input

       "bank" in "river bank" → different bank_input (different PE) →
       different Q, K, V than "bank" in "deposit bank"!

  5. Attention score matrices (seq_len × seq_len)
  6. Updated token representations after each layer
```

Does Every Vocab Token Get Processed?

```
Vocabulary: 50,257 tokens
Sentence:   "The river bank was flooded" = 5 tokens

Layer 1 processes:  5 vectors only
Layer 2 processes:  5 vectors only
...
Layer 12 processes: 5 vectors only

The other 50,252 tokens remain idle in the embedding matrix.
They are only activated when they appear in a sentence.

During training:
  Gradient updates ONLY reach the embedding rows of tokens
```

```
  that appeared in the current batch.
  All other rows are untouched for that batch.
  Over billions of training examples, every token
  eventually receives updates many times.
```

---

13. Training: Next Token Prediction

The Training Task

Transformers are trained with a deceptively simple objective: **given the context so far, predict the next token.** No labels needed — the text itself provides supervision.

```
Training text: "The cat sat on the mat"

Automatically generates multiple training pairs:

  Input: "The"                    → Target: "cat"
  Input: "The cat"                → Target: "sat"
  Input: "The cat sat"            → Target: "on"
  Input: "The cat sat on"         → Target: "the"
  Input: "The cat sat on the"     → Target: "mat"

One sentence → 5 training pairs for free!
With a trillion tokens of training data → countless examples.
```

Causal Masking (Decoder-only Models: GPT, Claude)

When predicting token at position t, the model must not see tokens at positions t+1, t+2, ... (they don't exist at inference time).

```
When predicting "on" (position 3):

  Can see:    "The" "cat" "sat" "on"    ← past and current
  Cannot see: "the" "mat"               ← future blocked!

Masking is applied to the attention score matrix:
  Set future scores to ∝'∝ž before softmax.
  After softmax: ∝'∝ž → 0 (no attention paid to future).

Attention mask (1=attend, 0=blocked):

              The  cat  sat  on  the  mat
  The:       [ 1    0    0   0    0    0 ]
  cat:       [ 1    1    0   0    0    0 ]
  sat:       [ 1    1    1   0    0    0 ]
  on:        [ 1    1    1   1    0    0 ]
  the:       [ 1    1    1   1    1    0 ]
  mat:       [ 1    1    1   1    1    1 ]

Training mirrors inference: at inference time future doesn't exist,
so training must never peek at future tokens either.
```

What Gets Trained (Everything, End-to-End)

```
OLD APPROACH (Word2Vec + RNN):
  Step 1: Train Word2Vec → freeze embeddings
  Step 2: Feed frozen embeddings into RNN → train RNN
  Problem: embeddings and RNN weights CANNOT co-adapt!

NEW APPROACH (Transformer):
  Train everything simultaneously:
```

```
    ✓ Embedding matrix
    ✓ All W_Q, W_K, W_V (every head, every layer)
    ✓ All W_O output projections
    ✓ All W1, W2 feed-forward weights
    ✓ All LayerNorm Î³, Î² parameters
    ✓ Output prediction weights

  Embeddings adjust to help the attention mechanism.
  Attention adjusts to use the embeddings optimally.
  Everything co-adapts. Far more powerful!
```

The Training Loss

```
After a full forward pass, the LAST TOKEN's representation is used:

  was_final = [0.82, 0.41, 0.93, ...]   ← 768-dim contextual vector

Why the last token?
  Due to causal masking, "was" has attended to ALL previous tokens.
  "was" carries the most complete summary of the context.
  "The" only attended to itself.
  "river" attended to "The" and itself.
  "bank" attended to "The", "river", "bank".
  "was" attended to ALL FOUR. Most complete!

Project to vocabulary:
  logits = W_out @ was_final   → 50,257 raw scores
  probs  = softmax(logits)     → 50,257 probabilities

Loss:
  L = âˆ’log P("flooded" | "The river bank was")
    = âˆ’log(probability assigned to the correct next token)

Goal: make this probability as high as possible.
Gradient flows backward through all 12 layers.
All weights update. Repeat for billions of examples.
```

---

14. Text Generation: The Autoregressive Loop

Step-by-Step Generation

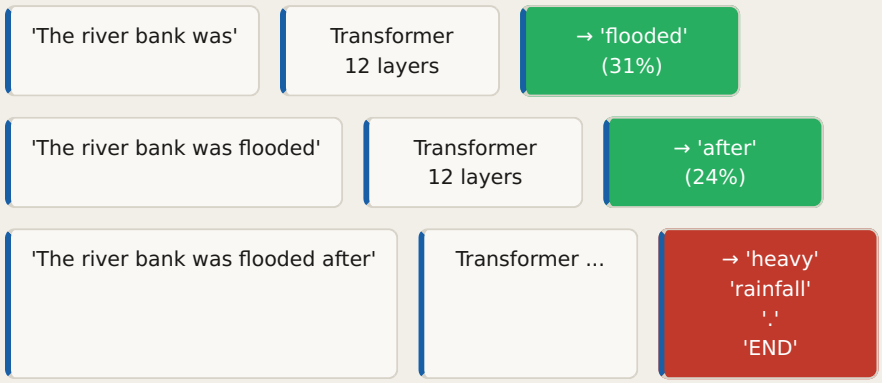
Step 1 — Forward pass, use last token:

```
Input: "The river bank was"
After 12 transformer layers...

was_final = [0.82, 0.41, 0.93, ...]

Project to vocabulary → 50,257 scores:
  "the":      2.1
  "a":        1.8
  "flooded":  8.7  ← highest!
  "dried":    6.2
  "empty":    4.1

Softmax → probabilities:
  "flooded":   31.2%
  "dried":      8.9%
  "destroyed":  6.1%
  "empty":      3.1%
  ...
```

Step 2 — Select next token:

```
Option 1 — Greedy decoding:
  Always pick the highest probability.
  → "flooded" every time. Deterministic, but repetitive.

Option 2 — Sampling:
  Randomly sample according to the probability distribution.
  → Usually "flooded" but sometimes "dried" or "empty".
  Adds variety and creativity to generated text.

Option 3 — Beam search:
  Track the top-k sequences simultaneously.
  Pick the sequence with highest combined probability.
  Used in translation for accuracy.
```

Step 3 — Autoregressive loop:



FLOW

'The river bank was' → Transformer

Transformer → → 'flooded'

→ 'flooded' → 'The river bank was flooded'

'The river bank was flooded' → Transformer

Transformer → → 'after'

→ 'after' → 'The river bank was flooded after'

'The river bank was flooded after' → Transformer ...

Transformer ... → → 'heavy'

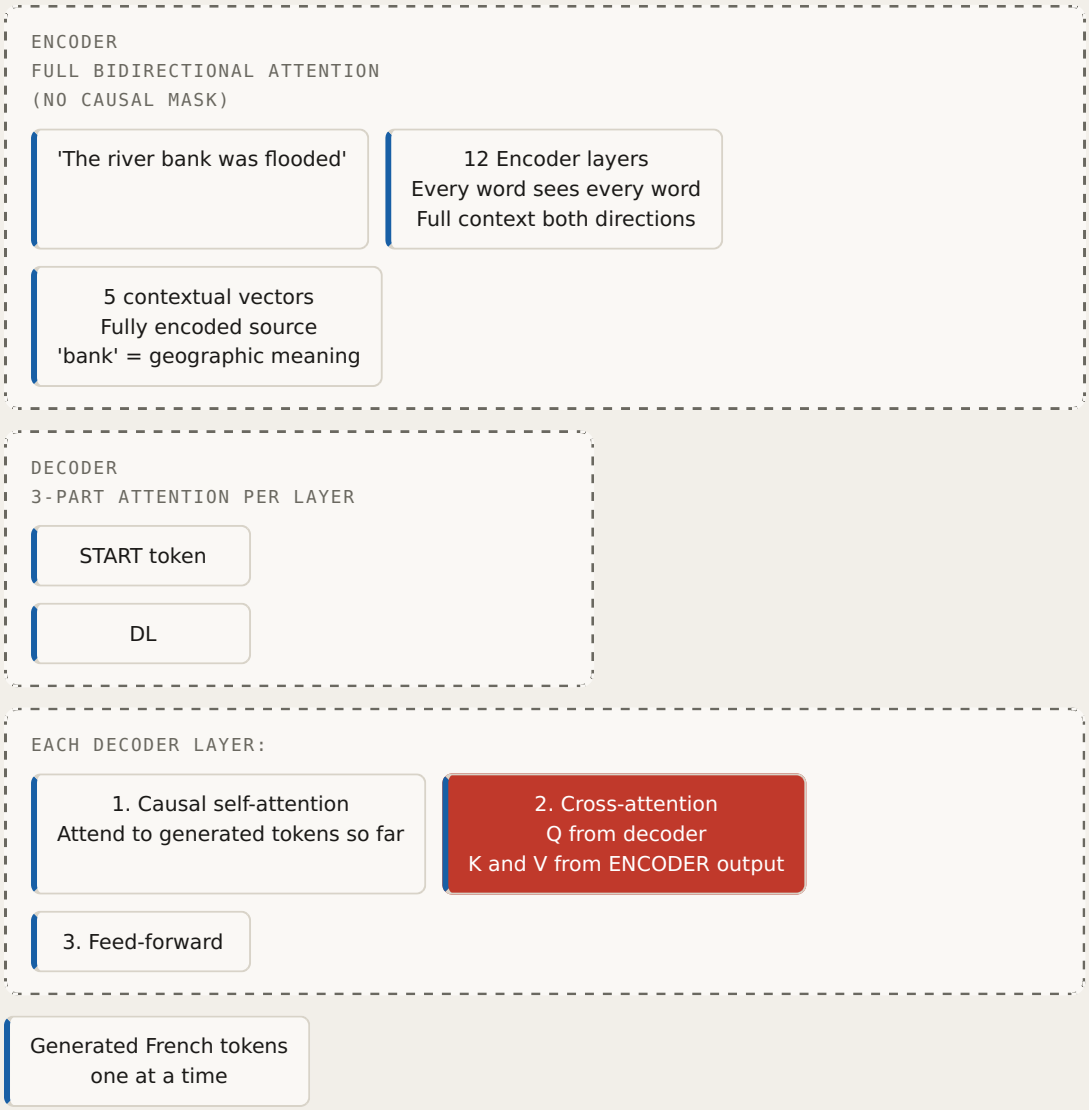
```
Iteration 1:  "The river bank was"              → "flooded"
Iteration 2:  "The river bank was flooded"      → "after"
Iteration 3:  "...flooded after"                → "heavy"
Iteration 4:  "...after heavy"                  → "rainfall"
Iteration 5:  "...heavy rainfall"               → "."
Iteration 6:  "...rainfall."                    → <END>

Final output: "The river bank was flooded after heavy rainfall."

Key: each generated token is appended to the input for the next step.
This is called AUTOREGRESSIVE generation.
```

15. Translation: Encoder-Decoder Architecture

When Is Encoder-Decoder Used?

```
Decoder-only (GPT, Claude, LLaMA):
  Process and generate text in the same language/space.
  Great for: chat, summarization, code, Q&A.
  The decoder attends to its own previous output (causal mask).

Encoder-Decoder (original Transformer, T5, MarianMT):
  Encoder FULLY understands the source.
  Decoder GENERATES the target language, attending to the encoder.
  Great for: machine translation, where the source must be
             fully digested before generation begins.
```

Architecture Overview



FLOW

'The river bank was flooded' → 12 Encoder layers

12 Encoder layers → 5 contextual vectors

START token → DL

1. Causal self-attention → 2. Cross-attention

2. Cross-attention → 3. Feed-forward

DL → Generated French tokens

5 contextual vectors *[Provides K and V for cross-attention]* → DL

Cross-Attention: The Bridge Between Languages

Cross-attention is the key mechanism that ties the encoder and decoder together:

```
In cross-attention (inside each decoder layer):
  Q (Query)  ← comes from the DECODER     "What French word am I generating?"
  K (Key)    ← comes from the ENCODER     "Which English words are available?"
  V (Value)  ← comes from the ENCODER     "Here is the English content."
```

```
This means: the decoder searches through the source sentence
            to find what it needs for each target word.
```

Generating "berge" (French for "river bank"):

```
Decoder has generated "La" so far.
Now generating "berge":

Q_berge matches against all encoder Keys:
  score(berge → "The")      = 0.12  ← low
  score(berge → "river")    = 0.31  ← medium (geographic)
  score(berge → "bank")     = 0.62  ← HIGH — this is what we're translating!
  score(berge → "was")      = 0.08  ← low
  score(berge → "flooded")  = 0.18  ← some (flood context)

Weighted sum of encoder Values:
  new_berge = 0.62×V_bank + 0.31×V_river + 0.18×V_flooded + 0.12×V_The + ...

"berge" now carries the English semantic content of "bank" in this context!
It knows "bank" means geographic here (from the encoder's disambiguation).
Output: "berge" ✓
```

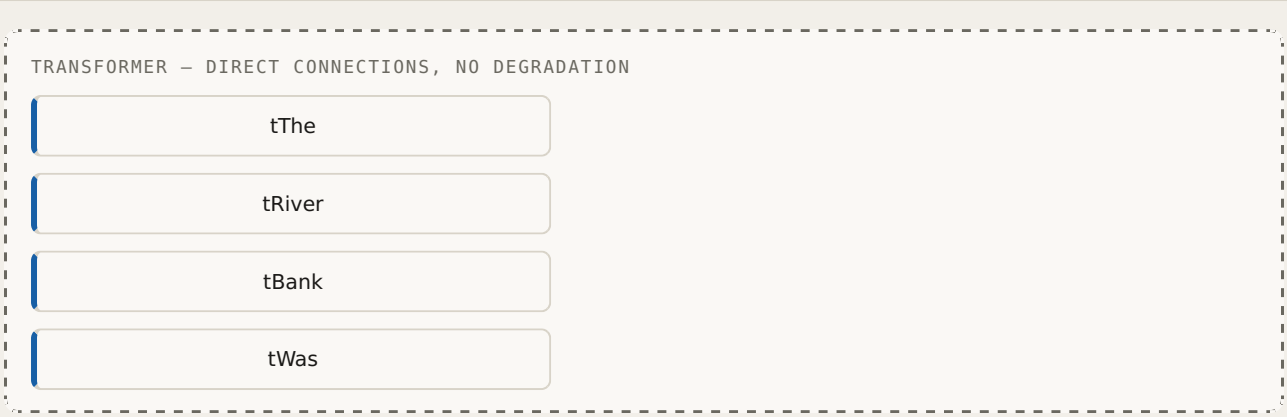
Full Translation Example

```
Source: "The river bank was flooded"
Target: "La berge de la riviÃ¨re était inondée"

ENCODER (full attention, no mask):
  All 5 English words see all 5 English words.
  "bank" fully disambiguated as geographic.
  Output: 5 rich contextual vectors.

DECODER (generates one token at a time):
  START → cross-attends to "The" (0.8) → "La"
  "La" → cross-attends to "bank"(0.6), "river"(0.3) → "berge"
  "berge" → cross-attends to "of/the" context → "de"
  "de" → cross-attends to "river" (0.7) → "la"
  "la" → cross-attends to "river" (0.8) → "riviÃ¨re"
  "riviÃ¨re" → cross-attends to "was" (0.6) → "était"
  "était" → cross-attends to "flooded" (0.9) → "inondée"
  "inondée" → END

Final: "La berge de la riviÃ¨re était inondée" ✓
```

---

16. RNN vs Transformer: Final Comparison

Information Flow



Feature-by-Feature Comparison

FEATURE	RNN	LSTM	TRANSFORMER
Processing order	Sequential (slow)	Sequential (slow)	Parallel (fast!)
Vanishing gradients	Severe	Solved via cell state	Not applicable
Long-range dependencies	Poor	Better	Excellent
Context used	Past only	Past only	Past + future
"bank" disambiguation	Partial	Better	Full
GPU parallelism	None	None	Full
Gradient highway	None	Cell state	Residual connections
Context window	Full sequence	Full sequence	Full sequence

Gradient Flow Comparison

```
RNN:
  Loss → h50 → h49 → ... → h1 → W_h
  Each step multiplies by (W_h × tanh') ≈ 0.4
  After 50 steps: (0.4)^50 ≈ 10â»²â°   VANISHES!

Transformer:
  Loss → Layer 12 → Layer 11 → ... → Layer 1 → Embeddings
         ↑ residual bypass at every layer
  ∂output/∂input = 1 + ∂(correction)/∂input
  The constant 1 prevents vanishing.
  Gradient flows cleanly through all 12 layers.
```

---

17. Key Formulas Quick Reference

Positional Encoding

```
PE(pos, 2i)   = sin( pos / 10000^(2i/d_model) )
PE(pos, 2i+1) = cos( pos / 10000^(2i/d_model) )

x_input = embedding + PE   ← element-wise addition
PE size  = d_model         ← NOT vocabulary size!
```

Self-Attention

```
Q = W_Q × x                          query transform
K = W_K × x                          key transform
V = W_V × x                          value transform

scores          = Q × Káµ€             raw attention scores
scaled_scores   = Q × Káµ€ / √d_k     scale to prevent collapse
attention_weights = softmax(scaled)  normalize per row (sums to 1)
output          = attention_weights × V   weighted content mix

Full formula:
Attention(Q, K, V) = softmax( Q × Káµ€ / √d_k ) × V
```

Multi-Head Attention

```
head_i   = Attention(W_Qi × x,  W_Ki × x,  W_Vi × x)
MultiHead = Concat(head_1, ..., head_h) × W_O
```

Complete Transformer Layer

```
# Attention block
x = LayerNorm( x + MultiHeadAttention(x) )

# Feed-forward block
x = LayerNorm( x + FeedForward(x) )

# Feed-forward definition
FeedForward(x) = W2 × ReLU(W1 × x + b1) + b2
```

Training Loss

```
L = ∝'log P(correct_next_token | all_previous_tokens)
  = cross-entropy loss over the vocabulary
```

Model Sizes Reference

```
Model         Layers  Heads  d_model  d_ff   Parameters
────────────  ──────  ─────  ───────  ─────  ──────────
BERT-base       12     12     768     3072    110M
GPT-2           12     12     768     3072    117M
GPT-2 XL        48     25    1600     6400   1.5B
GPT-3           96     96   12288    49152   175B
```

---

18. Common Confusions Clarified

PE Size = d_model, NOT Vocabulary Size

```
Vocabulary size (50,257) = number of ROWS in embedding matrix
                         = how many distinct tokens the model knows

d_model (768)            = number of DIMENSIONS per token vector
                         = the width of each embedding

Positional encoding size = d_model (768)

Reason: PE is added element-wise to embeddings.
  embedding = [v₁, v₂, ..., v₇₆₈]   768 values
  PE        = [p₁, p₂, ..., p₇₆₈]   768 values  ← must match!
  x_input   = [v₁+p₁, ..., v₇₆₈+p₇₆₈]

Adding 768 values to 50,257 values is mathematically impossible.
```

Q, K, V Roles (Very Commonly Confused)

```
Q (Query):  The token ASKING for information.
            "What context do I need to understand myself?"
            Created by the current token using W_Q.

K (Key):    Every token ADVERTISING what it is.
            "What kind of information do I hold?"
            Used to be matched against queries.

V (Value):  Every token's ACTUAL CONTENT.
            "Here is my semantic information to share."
            Retrieved and weighted, but does NOT control attention strength.

STRENGTH of attention = dot(Q, K) → softmax → attention weight
CONTENT retrieved     = weighted sum of V

V is not the strength. V is what flows when attention IS strong.
```

PE Is Added Once, Not at Every Layer

```
PE is added to embeddings ONCE, before Layer 1.
It is NOT re-added at each subsequent layer.

Positional information flows through the network via residual connections:
```

```
  Layer 1 output = Layer 1 input + correction
                 = (embedding + PE) + correction

The PE signal persists in the residual stream through all 12 layers.
```

Only Current Sentence Tokens Are Processed

```
Even though the embedding matrix contains 50,257 token rows,
only the tokens IN THE CURRENT SENTENCE flow through the
transformer layers.

5-token sentence → 5 vectors processed per layer.
The other 50,252 vocabulary tokens are untouched.
```

W_x in RNN â‰ Embedding Matrix

```
Embedding matrix (Word2Vec W1 / transformer embedding):
  Input:  token ID (integer)
  Output: embedding vector (e.g. 768-dim)
  Role:   "dictionary" — look up a word's meaning vector

W_x in RNN:
  Input:  embedding vector (e.g. 300-dim)
  Output: contribution to hidden state (e.g. 256-dim)
  Role:   "translator" — project the embedding into RNN hidden space

They appear at different stages, have different shapes,
and serve completely different purposes.
```

Static Embeddings vs Contextual Representations

```
Static (Word2Vec, embedding matrix lookup):
  "bank" always → [0.25, 0.47, 0.38, ...]
  Same vector regardless of context.

Contextual (transformer output after all layers):
  "bank" in "river bank"   → [0.91, 0.13, 0.82, ...]
  "bank" in "deposit bank" → [0.15, 0.87, 0.31, ...]
  Created dynamically for each sentence.

The embedding matrix gives "bank" the same STARTING vector.
The transformer layers transform it into a context-specific FINAL vector.
```

This document, together with the RNN & Word2Vec reference guide, forms a complete foundation for understanding modern Large Language Models from embeddings through generation.

PART IV

The nanochat Build

The hands-on portion. Built from Karpathy's nanochat on top of PyTorch. You're here right now — Phase 3.1 (Q, K, V projections) is your current stopping point.

> All code blocks are annotated: `[PT]` for PyTorch built-ins, `[NC]` for nanochat custom. Shape traces use the convention `(B, T, C)`.

CURRENT STOPPING POINT

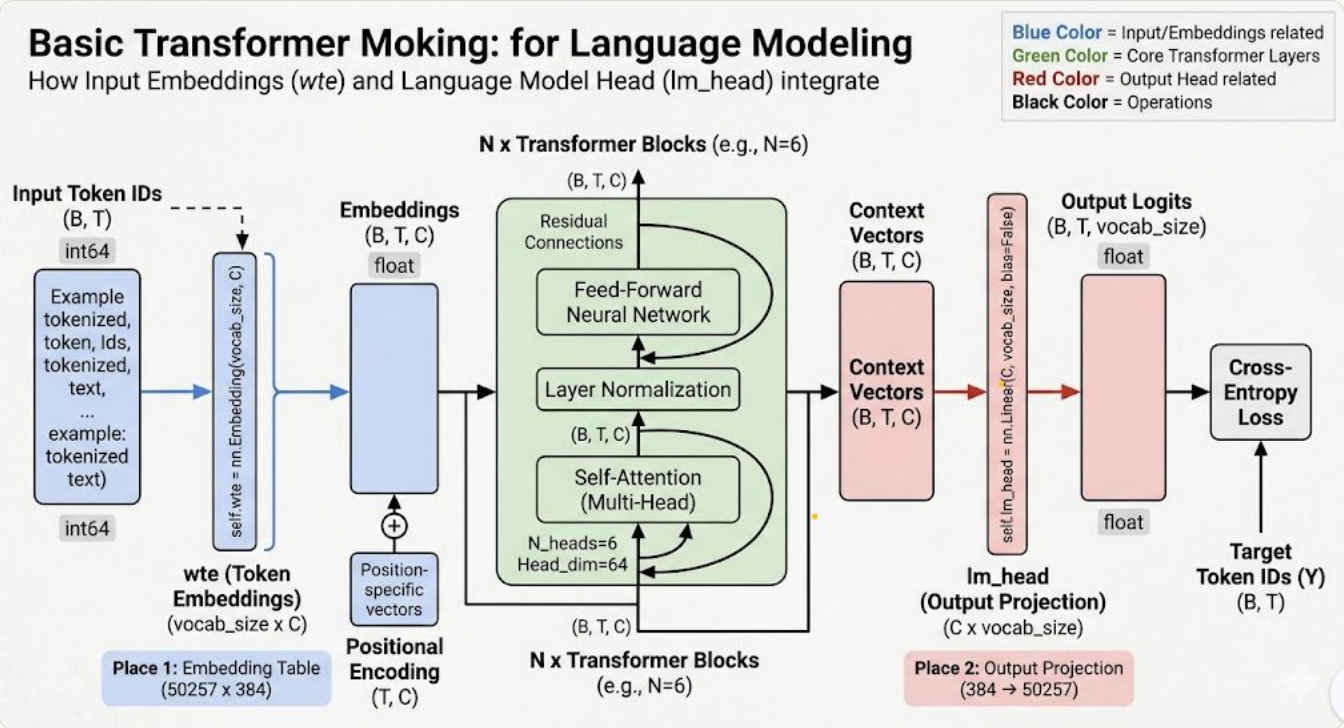
You are here — end of Phase 3.1 (Q, K, V projections)

Phases completed: Tokenisation (1) · Embeddings & infra (2) · Q, K, V projections (3.1).
Still ahead: **3.2 attention scores** · **3.3 causal mask** · **3.4 multi-head** · **4 transformer block** · **5 training & generation**. One more phase remains after 3 before the build week wraps.

Overview

The Full nanochat Pipeline

Every phase in this journal maps to one region of this diagram



Colour key

COLOUR	REGION	COVERED IN
■ Blue	Input token IDs → wte embedding → positional encoding	Phase 1 + Phase 2
■ Green	N × Transformer Blocks — attention + LayerNorm + FFN + residuals	Phase 3 + Phase 4
■ Red	lm_head output projection → logits → cross-entropy loss	Phase 5

The shape trace — left to right across the diagram

get_batch() → (B, T) int64 raw token IDs from train.bin wte(idx) → (B, T, C) float32 each integer → 384-dim vector [Entrance] + wpe(pos) → (B, T, C) float32 vectors now know their position × N blocks → (B, T, C) float32 same shape, deeply richer meaning [Thinking] lm_head(x) → (B, T, V) float32 one logit per possible next token [Exit] cross_entropy → scalar float32 single number minimised during training

> **Naming convention throughout this document**
> B = batch_size (12) · T = block_size / sequence length (1024) · C = n_embd (384) · V = vocab_size (50,257) · N = n_layer (6) · d_h = head_dim = C/n_head = 64
> **C and n_embd are identical.**
> C is used in shape traces (shorter). n_embd appears in GPTConfig and code.

The two places vocab_size appears (Place 1 and Place 2 in the diagram)

Place 1 — wte (Entrance)

`nn.Embedding(vocab_size, n_embd)` PT

Shape: (50,257 × 384) = 19.3M parameters

One learnable row per vocabulary token.

Integer ID → 384-dim float vector.

Place 2 — lm_head (Exit)

`nn.Linear(n_embd, vocab_size, bias=False)` PT

Shape: (384 → 50,257)

One output score per vocabulary token.

Highest logit = predicted next token.

> **Weight tying — wte and lm_head share the same matrix**
>
> `self.lm_head.weight = self.transformer.wte.weight` NC
>
> The entrance and exit share parameters. The same "meaning geometry" that encodes tokens also scores their likelihood. Halves parameter count at these two large layers.

Key Terms

wte · lm_head · n_head

The Entrance, Exit, and Thinking Style of the model

?

The intuition

Computers can't do math on words. Raw token IDs like `50256` are just arbitrary integers — the number itself carries no meaning. Three architectural elements bridge the gap between text and computation.

wte — The Entrance

A giant lookup table. Token ID → 384 floats encoding meaning.

Shape: (50,257 × 384)

Params: 19.3M learnable

ID 2746 → row 2746 → [0.21, −0.54, 0.88, ...]

Pure lookup — no matrix multiply.

lm_head — The Exit

Projects 384-dim context back to 50,257 scores.

Shape: (384 → 50,257)

Output: one logit per vocab token

Highest logit = predicted next token.

Shares weights with wte (weight tying).

n_head — Thinking Style

Divides 384 dims into 6 specialist perspectives.

Split: 384 ÷ 6 = 64 dims per head

Pattern: view() → transpose() → parallel attention

Each head learns its own focus. Results concatenated back to 384.

▦

Visual — n_head specialisation

THE PHOTO ANALOGY — LOOKING AT "A CAT SITTING ON A MAT"

With 1 head: look at everything at once — trying to understand colour, shape, relationships, text all simultaneously. Limited capacity, no specialisation.

With 6 heads (each sees only 64 of 384 dims):

```
Head 0 → grammar relationships?
Head 1 → topic / subject matter?
Head 2 → nearby context?
Head 3 → coreference ("it" = what)?
Head 4 → long-range dependencies?
Head 5 → positional patterns?
```

> **The constraint that creates specialisation**
>
> Each head only sees 64 of the 384 dimensions. It *cannot* try to track everything — it is forced to develop a focused perspective. Scarcity is what makes the heads diverge. The model learns what each head specialises in — you never assign roles manually.

★

Summary table

TERM	WHAT IT IS	ANALOGY	SHAPE	PYTORCH
wte	Token embedding table	Dictionary: ID → coordinates	(50257, 384)	PT nn.Embedding
lm_head	Output projection	Voting booth: pick next word	(384, 50257)	PT nn.Linear
n_head	Attention head count	Team of specialists	splits C → 6 × 64	NC split logic

Output

Logits, Softmax, and Cross-Entropy Loss

The output pipeline — from context vectors to training signal

?

The problem

After the transformer blocks finish processing, each token position holds a 384-dim vector. But the model's job is to predict the *next token* from 50,257 possibilities. Three operations convert that vector into a training signal.

▦

Visual — the three-step pipeline

① Logits — raw scores

Output of lm_head. Not probabilities — just rankings. Can be any value, positive or negative.

"cat" → 8.7 "sat" → 6.2 "the" → 2.1 "jumped" → −1.4

Negative = unlikely, not impossible. Don't sum to anything meaningful yet.
② Softmax — probabilities
Three steps: **exp()** → **sum** → **divide**.
Result sums to exactly 1.0.

exp(8.7) = 6002.9
exp(6.2) = 492.7
sum = 6505.4

"cat" → 6002.9/6505.4 = **92.3%**
Ranking is preserved. Only scale changes.
③ Cross-entropy — loss
loss = −log(prob of correct token)

95% correct → −log(0.95) = **0.05** ✓
50% correct → −log(0.50) = **0.69**
1% correct → −log(0.01) = **4.61** ✗
0% correct → −log(0.00) = ∞

One scalar. Flows backward through every layer during training.
{ }
Implementation

```
# ── TRAINING ──────────────────────────────────────────────────────
logits = self.lm_head(x)             PT  # (B, T, 50257) raw scores

loss = F.cross_entropy(              PT  # fused log-softmax + numerically stable
    logits.view(-1, vocab_size),     NC  # (B*T, 50257) — flatten for PyTorch
    targets.view(-1)                 NC  # (B*T,)       — flatten targets
)
# Returns one scalar — average loss across all B*T token predictions
loss.backward()   PT  # nudge every weight to reduce this number

# ── INFERENCE ─────────────────────────────────────────────────────
last_logits = logits[:, -1, :]           NC  # (B, 50257) — last position only
probs = F.softmax(last_logits, dim=-1)   PT  # probabilities summing to 1

# Option A — greedy (deterministic, always picks highest)
next_token = probs.argmax(dim=-1)        PT

# Option B — sampling (creative, weighted random)
next_token = torch.multinomial(probs, 1)  PT

idx = torch.cat([idx, next_token], dim=1) PT  # append and loop
```

★
Key insights

Never softmax before F.cross_entropy

F.cross_entropy applies log-softmax internally in a fused, numerically stable operation. Softmaxing first produces numbers like 0.9999997 — log() loses floating-point precision and errors accumulate across B×T predictions.

Training vs inference — the key difference

Training: use ALL T positions simultaneously, compare against targets, compute loss.
Inference: use only the LAST position's logit (`logits[:, -1, :]`), no targets, softmax to pick next token.

The causal mask preview

Attention scores are (T, T) pairwise relevance ratings. The causal mask sets scores above the diagonal to −∞ before softmax — token at position t cannot attend to positions t+1, t+2, ... After softmax, −∞ becomes zero probability. Future tokens contribute nothing.

Phase 1

Tokenisation

How raw text becomes integer tensors the model can process

A neural network can only work with numbers. The tokeniser is the bridge — it sits entirely outside the PyTorch model, has no trainable parameters, and runs once before training begins.

"Hello world" → [15496, 995] → nn.Embedding → (2, 384) float tensor

1.1 — BPE (Byte Pair Encoding)

?

The problem — three bad options

STRATEGY	VOCAB SIZE	"UNHAPPINESS"	CONTEXT EFFICIENCY	PROBLEM
Character-level	65	11 tokens	~930 chars / 1024 T	Wastes context window
BPE subword ★	**50,257**	**3 tokens**	**~4000 chars / 1024 T**	**Best of both worlds**
Word-level	millions	1 token	great	Unknown words get no embedding

BPE hits the sweet spot — small enough vocabulary for efficient embeddings, large enough tokens to pack real meaning into the context window. A 1024-token window holds ~930 characters at char-level vs ~4000 at BPE — a 4× win.

▦

Visual — the algorithm running step by step

BPE is greedy statistics — nothing more. Start with characters. Find the most frequent adjacent pair. Merge it. Repeat until vocabulary target is hit.

CORPUS: "LOW"(×5) · "LOWER"(×2) · "NEWEST"(×6) — WITH END-OF-WORD MARKER Ġ

STEP 0 — SPLIT INTO CHARACTERS

l o w Ġ·l o w e r Ġ·n e w e s t Ġ

lowernstĠ 9 tokens — vocab size 9

COUNT ADJACENT PAIRS — FIND THE WINNER

PAIR	APPEARS IN	COUNT
l + o	"low", "lower"	**2 ← winner**
o + w	"low", "lower"	2 (tie — "l+o" wins first)
w + e	"lower", "newest"	2
e + s	"newest"	1

MERGE 1l + o→lo

l o w Ġ → **lo w Ġ**

l o w e r Ġ → **lo w e r Ġ**

lowernstĠlo 10 tokens

MERGE 2lo + w→low

lo w Ġ → **low Ġ**

lo w e r Ġ → **low e r Ġ**

lowernstĠlolow 11 tokens

After thousands of merges on real web text, unseen words still tokenise cleanly:

"lowering" → loweringĠ — shares **low** with "low", "lower", "lowest". Meaning shared by construction.

{ }

From-scratch Python implementation

```
# [NC] All code below — plain Python, no PyTorch needed
from collections import Counter

def get_pairs(vocab):
    """Count all adjacent pairs across the whole corpus."""
    pairs = Counter()
    for word, freq in vocab.items():
        symbols = word.split()
        for i in range(len(symbols) - 1):
            pairs[(symbols[i], symbols[i+1])] += freq
    return pairs

def merge_vocab(pair, vocab):
    """Replace winning pair everywhere in corpus."""
    new_vocab = {}
    bigram = ' '.join(pair)
    replacement = ''.join(pair)
    for word in vocab:
        new_vocab[word.replace(bigram, replacement)] = vocab[word]
    return new_vocab

vocab = {'l o w Ġ': 5, 'l o w e r Ġ': 2, 'n e w e s t Ġ': 6}
merge_rules = []          # this ordered list IS the tokeniser

for i in range(10):
    best  = max(get_pairs(vocab), key=get_pairs(vocab).get)
    vocab = merge_vocab(best, vocab)
    merge_rules.append(best)
    # merge_rules = [('l','o'), ('lo','w'), ('low','e'), ...]
```

```
# Encode new text: apply merge rules in order
def tokenise(text, merge_rules):
    words = [' '.join(list(w)) + ' Ġ' for w in text.split()]
    for (a, b) in merge_rules:
        words = [t.replace(f'{a} {b}', f'{a}{b}') for t in words]
    return [tok for t in words for tok in t.split()]
```

★

Key insight

> **The merge_rules list IS the tokeniser**
>
> Not the final vocabulary — the *ordered list of merges*. To encode new text you don't look up a dictionary; you apply each merge rule in order, transforming the character sequence step by step. Serialise this list to disk and you have a complete, reloadable tokeniser. This is exactly what tiktoken distributes — 50,000 merge rules compiled in Rust.

1.2 — tiktoken (what nanochat uses)

?

What tiktoken is

tiktoken is GPT-2's pre-trained BPE tokeniser — 50,000 merge rules trained by OpenAI on web text, shipped as a fast Rust library. You are *borrowing* their vocabulary, not training your own. The interface mirrors the char tokeniser: `encode()` and `decode()`.

	CHAR TOKENISER	TIKTOKEN GPT-2
Vocab size	65	50,257
Chars per token	1	~4
Context efficiency	poor	4× better
Vocab trained on	your corpus	OpenAI web text
Dependency	none	pip install tiktoken

{ }

Usage + data preparation

```
import tiktoken, numpy as np

enc = tiktoken.get_encoding("gpt2")       NC
# "gpt2" → 50,257 tokens · "cl100k_base" → GPT-4 (100k tokens)
# Downloads ~500KB merge rules once, caches in ~/.tiktoken

# encode / decode — same interface as char tokeniser
ids  = enc.encode_ordinary(text)          NC  # str → list[int]
text = enc.decode(ids)                    NC  # list[int] → str

# Inspect individual tokens (very useful for debugging):
# enc.decode([314]) → ' I'  ← note the leading space!
# Spaces attach to the following word in GPT-2's vocab

# ── prepare.py — run ONCE before training ──────────────
with open('data/input.txt') as f: data = f.read()
ids = enc.encode_ordinary(data)           NC
```

```
n   = int(0.9 * len(ids))
np.array(ids[:n], dtype=np.uint16).tofile('data/train.bin')  NC
np.array(ids[n:], dtype=np.uint16).tofile('data/val.bin')    NC
# uint16 = 2 bytes/token vs int64 = 8 bytes → 4× smaller files
# Values 0..50256 fit in uint16 (max 65535)

# ── get_batch() — loads from .bin at training time ──────────
train_data = np.memmap('data/train.bin', dtype=np.uint16, mode='r')  NC

def get_batch(split):                                          NC
    data = train_data if split == 'train' else val_data
    ix   = torch.randint(len(data) - block_size, (batch_size,))   PT
    x    = torch.stack([torch.from_numpy(                          PT
               data[i  :i+block_size  ].astype(np.int64)) for i in ix])
    y    = torch.stack([torch.from_numpy(                          PT
               data[i+1:i+block_size+1].astype(np.int64)) for i in ix])
    return x.to(device), y.to(device)                             PT
# x: (B, T) int64   y: (B, T) int64  — same shape, y is x shifted right by 1
# .astype(np.int64): files stored as uint16 (compact), nn.Embedding needs int64
```

★

Key insights

encode_ordinary vs encode — important distinction

Use `encode_ordinary()` for data prep — treats <|endoftext|> as plain text. Use `encode()` with `allowed_special` when inserting EOT tokens between documents. Mixing them silently produces different token IDs for the same text.

The training signal — x and y shifted by 1

Position 0: given [F], predict i · Position 1: given [F,i], predict r · …

One sequence of 1024 tokens gives 1024 independent training examples — all computed in a single forward pass. The causal mask ensures each position only sees its own past.

1.3 — Character-level tokeniser (Shakespeare demo)

{ }

Complete implementation — 15 lines of plain Python

```
# [NC] entirely — plain Python, no PyTorch until the tensor conversion
import torch                                          PT

with open('shakespeare.txt', 'r', encoding='utf-8') as f:
    text = f.read()

chars      = sorted(list(set(text)))  NC  # 65 unique chars in Shakespeare
vocab_size = len(chars)               NC  # sorted() = deterministic mapping across runs

stoi = { ch:i for i,ch in enumerate(chars) }  NC  # string → int
itos = { i:ch for i,ch in enumerate(chars) }  NC  # int → string

encode = lambda s: [stoi[c] for c in s]           NC
decode = lambda l: ''.join([itos[i] for i in l])  NC

# PyTorch enters only for tensor conversion:
data = torch.tensor(encode(text), dtype=torch.long)   PT
# shape: torch.Size([1115394]) — the full Shakespeare corpus as integers

n          = int(0.9 * len(data))        NC
```

```
train_data = data[:n]                    NC
val_data   = data[n:]                    NC
```

Why sorted()?

`set()` gives a random order each run — without `sorted()`, the same character maps to a different integer every time. The model would load saved weights and map them to the wrong characters. Determinism is essential for reproducibility.

Phase 2

Embeddings

How integer token IDs become float vectors, and how the model knows word order

After Phase 1, `get_batch()` returns `idx` of shape **(B, T)** — a grid of integers. Integers are arbitrary IDs. The number 2746 has no mathematical relationship to 2747. You can't do meaningful matrix multiplication on raw integers. Embeddings fix this — converting each integer into a 384-dimensional float vector where proximity encodes meaning.

2.1 — idx: what it is and what it contains

▦

Visual — idx is just a grid of integers

IDX SHAPE: (B=3, T=8) — EACH CELL IS A TOKEN ID

```
idx = tensor([
    [18, 47, 56, 57, 58,  1, 15, 47],   NC  # sentence 1: "First Ci…"
    [39, 44, 42, 53, 56,  1, 58, 46],   NC  # sentence 2: "before t…"
    [ 1, 61, 47, 50, 50,  1, 58, 46],   NC  # sentence 3: " will th…"
])
# dtype: int64   shape: (B=3, T=8)
# Every cell is just an integer — a row number into the embedding table
```

`idx` lives for exactly *one line* in `forward()`. It enters as integers, `wte` converts it to vectors, and from that point on everything is called `x` and operates in float vector space. `idx` is never used again.

2.2 — nn.Embedding: the lookup table (wte)

?

What it is

`nn.Embedding` is a matrix with a special forward pass: instead of matrix multiply, it does a **row lookup**. Give it an integer, get back that row. That is the entire operation — no computation, pure lookup.

▦

Visual — the embedding matrix

WTE SHAPE: (50257, 384) = 19.3M LEARNABLE PARAMETERS

ROW (TOKEN ID)	TOKEN	384-DIM VECTOR (FIRST 6 SHOWN)
0	"!"	[0.12, −0.45, 0.33, 0.71, −0.22, 0.08, …]
257	" " (space)	[0.02, 0.11, −0.03, 0.05, 0.01, −0.09, …]

ROW (TOKEN ID)	TOKEN	384-DIM VECTOR (FIRST 6 SHOWN)
2746	**"model"**	**[0.21, −0.54, 0.88, 0.33, −0.71, 0.42, …] ← idx=2746 returns this row**
15496	"Hello"	[0.55, 0.23, −0.18, 0.62, 0.31, −0.44, …]
50256	<EOT>	[−0.31, 0.07, 0.55, −0.88, 0.14, 0.66, …]

The 384 numbers are *static* embeddings — "bank" always maps to the same row regardless of context ("river bank" or "bank deposit"). The transformer blocks above transform it into a context-specific vector. The embedding table is the starting point, not the final answer.

After training: `"cat"` and `"dog"` are nearby in 384-dim space. Mathematical relationships emerge: `king − man + woman ≈ queen`. These are not programmed in — they fall out of the geometry that training creates.

{ }

Implementation

```
self.wte = nn.Embedding(config.vocab_size, config.n_embd)  PT
# Creates matrix of shape (50257, 384)
# Initialised randomly — every value learned during training
# 50257 × 384 = 19.3M learnable parameters

# Forward pass — called automatically as self.wte(idx):
tok_emb = self.wte(idx)   PT
# idx:     (B, T)       int64  — token IDs
# tok_emb: (B, T, 384)  float  — one row per token ID
# Internally: for each integer in idx, return embedding_matrix[integer]
# No matrix multiply. Pure lookup. Very fast.
```

2.3 — Positional encoding (wpe)

?

Why we need it

Self-attention processes all tokens simultaneously — it has no built-in sense of order. Without positional encoding, "cat bit dog" and "dog bit cat" produce *identical* attention patterns. The model literally cannot tell which word came first.

WHY NOT JUST ADD POSITION NUMBERS?

OPTION	APPROACH	PROBLEM
A — raw numbers	pos 2 → x + 2 = [0.25+2, ...]	Values explode at pos 100+
B — normalise 0–1	pos 2 in 10-word = 0.20, in 100-word = 0.02	Same position, different value
C — learned vectors ✓	One 384-dim vector per position, trained	Stable, consistent, simple

{ }

wpe + combining with wte

```
self.wpe = nn.Embedding(config.block_size, config.n_embd)   PT
# shape: (1024, 384) — one row per position in context window
# ~393K parameters — much smaller than wte's 19.3M
# Row 0 = "I am the first token"  Row 1023 = "I am the 1024th token"

# Sinusoidal vs learned PE: original 2017 Transformer used fixed sine/cosine
```

```
# nanochat/GPT-2 uses learned PE — simpler, equally effective

pos     = torch.arange(T, device=idx.device)    PT  # (T,) → [0,1,2,…,T-1]
pos_emb = self.wpe(pos)                         PT  # (T, 384)

# Element-wise addition — combines token meaning + position info:
# wte("The") = [0.11, -0.33,  0.55, …]   ← what "The" means
# wpe(pos=0) = [0.14,  0.99,  0.28, …]   ← "I am first position"
#              ──────────────────────────
# sum        = [0.25,  0.66,  0.83, …]   ← "The" at position 0 ✓

x = self.transformer.drop(tok_emb + pos_emb)   PT
# tok_emb: (B, T, 384)  pos_emb: (T, 384)
# Broadcasting: PyTorch auto-expands pos_emb across the B dimension
# Same positional vector for every sentence in the batch — correct!
# PE is added ONCE here. Persists through all 6 blocks via residuals. Never re-added.
```

★

Key insight — broadcasting

> **How (T, 384) adds to (B, T, 384)**
>
> tok_emb is (B, T, 384) but pos_emb is only (T, 384). PyTorch's broadcasting rule auto-expands pos_emb to (B, T, 384) — the same positional vector is added to every item in the batch. This is correct: position 0 means the same thing in every sentence. One GPU operation, no Python loop.

2.4 — Inside every transformer block

▦

Visual — the two-half structure

Half 1 — Attention sublayer

`LayerNorm(384) PT`

↓

`CausalSelfAttention NC`

↓

`x = x + attn_output` *residual connection*

Half 2 — FFN sublayer (plain 2-layer NN!)

`LayerNorm(384) PT`

↓

`nn.Linear(384→1536) PT` *4× expand*

↓

`nn.GELU() PT` *activation*

↓

`nn.Linear(1536→384) PT` *compress*

↓

`x = x + ffn_output` *residual connection*

> **Shape is preserved through ALL 6 blocks**
>
> (B, T, 384) goes in → (B, T, 384) comes out — every single block. Each block enriches meaning without touching dimensions. The 4× FFN expansion (384→1536→384) provides working space for intermediate computation. GELU instead of ReLU: smooth fade near zero gives cleaner gradients for language models.

{ }

MLP class + complete GPT forward pass

```
class MLP(nn.Module):                    NC
    def __init__(self, config):
        super().__init__()               PT
        self.c_fc   = nn.Linear(         PT  # expand: 384 → 1536
            config.n_embd, 4 * config.n_embd)
        self.gelu   = nn.GELU()          PT  # activation
        self.c_proj = nn.Linear(         PT  # compress: 1536 → 384
            4 * config.n_embd, config.n_embd)

    def forward(self, x):             NC
        x = self.c_fc(x)              PT
        x = self.gelu(x)              PT
        x = self.c_proj(x)            PT
        return x

class GPT(nn.Module):                    NC
    def forward(self, idx, targets=None):  NC
        B, T = idx.shape                 NC

        # ── Phase 2: embeddings ──────────────────────────
        tok_emb = self.transformer.wte(idx)           PT  # (B,T,384)
        pos     = torch.arange(T, device=idx.device)  PT  # (T,)
        pos_emb = self.transformer.wpe(pos)           PT  # (T,384)
        x = self.transformer.drop(tok_emb + pos_emb)  PT  # (B,T,384)

        # ── Phase 3: transformer blocks ──────────────────
        for block in self.transformer.h:   NC
            x = block(x)                  NC  # (B,T,384)→(B,T,384)
        x = self.transformer.ln_f(x)      PT  # final LayerNorm

        # ── Phase 5: output ──────────────────────────────
        logits = self.lm_head(x)          PT  # (B,T,50257)
        loss = None
        if targets is not None:           NC
            loss = F.cross_entropy(        PT
                logits.view(-1, logits.size(-1)),
                targets.view(-1))
        return logits, loss
```

2.5 — LayerNorm, AdamW, and training infrastructure

▦

Visual — LayerNorm is NOT ±1 clipping

LAYERNORM ON EXAM SCORES [20, 40, 60, 80, 100]

Step 1 — subtract mean (60):

[−40, −20, 0, +20, +40]

Step 2 — divide by std (28.3):

[−1.41, −0.71, 0, +0.71, +1.41]

Score of 200 → (200−60)/28.3 = **+4.9**

No clipping — just consistent scale

The benefit is consistency.

Every vector entering the next sublayer has mean=0 and std=1. Attention weights don't have to cope with values that are sometimes 0.001 and sometimes 1000.

Training is dramatically more stable. Applied before attention and before FFN in every block (pre-norm pattern).

▦

Visual — SGD → Adam → AdamW

Plain SGD

weight = weight − lr × gradient

One fixed learning rate for all weights. Zigzags in ravines. Can't handle tiny and huge gradients simultaneously.

Adam (2015)

Tracks **m** (momentum) and **v** (variance) per weight.

update = lr × m / (√v + ε)

Every weight gets its own adaptive learning rate. **Bug:** weight decay gets rescaled by √v — loses effect.

AdamW (2017) ← nanochat

Adam + decay applied *after* gradient update — decoupled.

weight -= lr × m / (√v + ε) *gradient*

weight -= lr × λ × weight *decay, clean*

Every weight decays by consistent λ × weight regardless of gradient size.

ADAM INTERNALS — THE "M" AND "V" TRACKERS

m (momentum / exp_avg): Running average of recent gradient *directions*. β_1=0.9 means "90% old direction + 10% new gradient." Acts like a heavy ball — smooths noise, doesn't stop instantly at small bumps.

v (variance / exp_avg_sq): Running average of recent gradient *magnitudes squared*. β_2=0.95 means "95% old size + 5% new gradient²." Tracks volatility.

The automatic gearbox: Dividing by √v gives each weight its own effective step size. Neglected weight (tiny v) → large step (gets a push). Exploding weight (huge v) → tiny step (moves cautiously). You never manually tune per-layer learning rates.

{ }

AdamW + training loop

```
# AdamW parameter groups — weight matrices decay, biases/LayerNorm don't
decay_params    = [p for n,p in model.named_parameters() if p.dim() >= 2]  NC
no_decay_params = [p for n,p in model.named_parameters() if p.dim() < 2]   NC

optimizer = torch.optim.AdamW([                                           PT
    {'params': decay_params,    'weight_decay': 0.1},
    {'params': no_decay_params, 'weight_decay': 0.0},
], lr=3e-4, betas=(0.9, 0.95), eps=1e-8)

# The complete training loop:
for step in range(max_iters):                                              NC
    optimizer.zero_grad(set_to_none=True)                                 PT  # 1. clear grads
    logits, loss = model(x, y)                                            NC  # 2. forward
    loss.backward()                                                       PT  # 3. backward
```

```
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)   PT  # 4. clip
    optimizer.step()                                          PT  # 5. update
    # step() does per weight:
    #   m = 0.9×m + 0.1×grad           (momentum)
    #   v = 0.95×v + 0.05×grad²        (variance)
    #   weight -= lr × m/(√v + 1e-8)   (adaptive update)
    #   weight -= lr × 0.1 × weight    (weight decay, decoupled)
```

★

AdamW parameters + memory cost

SYMBOL	PYTORCH ARG	NANOCHAT VALUE	ROLE
lr	lr	3e-4	Base step size — most sensitive hyperparameter
β_1	betas[0]	0.9	Momentum: 90% old direction + 10% new gradient (exp_avg)
β_2	betas[1]	0.95	Variance: 95% old size + 5% new gradient² (exp_avg_sq)
λ	weight_decay	0.1	Forgetting pressure — pushes weights toward zero
ε	eps	1e-8	Prevents division by zero in √v + ε

The 4× memory cost of Adam — why training LLMs is expensive

For every weight: store (1) the weight, (2) gradient, (3) m, (4) v = 4× model size in VRAM.
7B model at float32: 7B × 4 × 4 bytes =
112 GB
just to hold training state.
Solutions: bfloat16 precision (÷2), 8-bit optimisers (÷4 for m and v), ZeRO sharding across GPUs.
nanochat at 10–85M params: fits comfortably under 2 GB.

Gradient clipping — the speed limiter

Between loss.backward() and optimizer.step(): `clip_grad_norm_(params, max_norm=1.0)`
If gradient vector magnitude exceeds 1.0, scale everything down proportionally. Direction preserved.
Magnitude tamed. Prevents one bad batch from destroying weeks of training.

Phase 3

Causal Self-Attention

The heart of the transformer — how tokens attend to each other

This is the most important phase in the build. Everything before it was setup. The transformer's power comes entirely from what happens here: every token looks at every other token, scores how relevant each one is, and uses those scores to pull in information from across the sequence.

3.1 — Q, K, V Projections

?

Why three matrices? Why not compare embeddings directly?

The token embedding already carries 384 numbers describing the token. Why not just compare those directly? Because the embedding carries *everything* about a token at once. You need three specialised, separate views of it — each answering a different question, via three separate learned weight matrices.

▦

Visual — Q, K, V roles

Q — Query

"What am I looking for?"

The question a token is asking. Scored against every other token's Key to produce attention scores. Determines how much each other token is attended to.

K — Key

"What do I advertise I contain?"

The catalogue label. Matched against Queries. **K determines whether a token gets attended to.** Can be highly findable (strong K) while sharing something different in V.

V — Value

"What do I actually give away?"

Content retrieved when attention is high. **V determines what flows** when a token is attended to. Completely separate from K — this is why the model can learn nuanced retrieval patterns.

> **The library analogy**
> You are the Query (searching "books about cats"). Each book's catalogue entry is a Key. The score = how well your query matches each key. The actual pages you read = the Value. A beautifully written book (high V) with a terrible catalogue entry (low K) will never be found. Q, K, V are separate for exactly this reason.

▦

Visual — projection dimensions + the fused trick

FOR A SINGLE TOKEN EMBEDDING X OF SHAPE (384,)

```
x (384) × W_Q (384 × 64) = Q (64)    ← query for this token
x (384) × W_K (384 × 64) = K (64)    ← key for this token
x (384) × W_V (384 × 64) = V (64)    ← value for this token


head_dim = n_embd // n_head = 384 // 6 = 64
```

> **nanochat fuses all three into one matrix (efficiency)**
> Instead of 3 separate matmuls: `self.c_attn = nn.Linear(384, 3×384, bias=False)` PT
> One big multiply → (B, T, 1152) → `.split(384, dim=2)` → q, k, v each (B, T, 384)
> GPUs prefer large parallel operations. One matmul is faster than three small ones. Conceptually identical.

▦

Visual — .view() and .transpose() — the head split

SPLITTING (B, T, 384) INTO 6 HEADS OF 64 DIMS — TWO MANDATORY STEPS

```
Start:           (B, T, 384)          ← 384 flat dims, heads mixed together
.view(B,T,6,64)  (B, T, 6, 64)         ← heads labelled, 3D→4D, no data copy
.transpose(1,2)  (B, 6, T, 64)         ← heads moved to dim 1 for batched matmul
```

.VIEW() — RELABELLING, NO DATA COPY

Like drawing dividing lines on a strip of paper. The 384 numbers are already in memory in a flat block. .view(B, T, 6, 64) just says "interpret the last dim as 6 groups of 64." Zero data movement. Instant.

.TRANSPOSE(1,2) — MOVE HEADS TO DIM 1

Swaps dim 1 (T) and dim 2 (n_head). After this, heads sit in dim 1 — exactly where PyTorch's batched matmul expects them. All 6 heads then run as a batch: (B × 6) independent operations in parallel.

AFTER THE SPLIT — EACH HEAD INDEPENDENTLY SEES ITS 64-DIM SLICE

Head 0: q[b,0,:,:] → (T, 64)
dims 0–63

Head 1: q[b,1,:,:] → (T, 64)
dims 64–127

Head 2: q[b,2,:,:] → (T, 64)
dims 128–191

Head 3: q[b,3,:,:] → (T, 64)
dims 192–255

Head 4: q[b,4,:,:] → (T, 64)
dims 256–319

Head 5: q[b,5,:,:] → (T, 64)
dims 320–383

Why two steps are mandatory — not one

.view() reads memory strictly left-to-right. It must come first to correctly label which 64 dims belong to which head. Only after correct labelling can .transpose() safely reorder dimensions. Jumping straight to .view(B, 6, T, 64) would scramble which data each head sees — wrong results, no error thrown.

Visual — q @ k.transpose(-2,-1) — the attention score computation

WHY TRANSPOSE K? (INNER DIMS MUST MATCH)

```
q:          (B, 6, T, 64)
k (wrong):  (B, 6, T, 64)   ← 64≠T ✗
k.T(-2,-1): (B, 6, 64, T)   ← 64=64 ✓

result:     (B, 6, T, T)   ← 64 cancels
att[b,h,i,j] = how much token i
               attends to token j
```

CONCRETE DOT PRODUCT (HEAD_DIM=3)

```
Q_sat = [0.1,  0.8,  0.4]
K_cat = [0.8,  0.3,  0.5]

score =
  (0.1×0.8) + (0.8×0.3) + (0.4×0.5)
=   0.08    +   0.24    +   0.20
= 0.52 ← "sat" attends to "cat" ✓

score(sat→The) = 0.20 ← less attention ✓
```

{ }

CausalSelfAttention.__init__ — fully annotated

```
class CausalSelfAttention(nn.Module):              NC
    def __init__(self, config):
        super().__init__()                         PT
        assert config.n_embd % config.n_head == 0  NC  # head_dim must be integer

        self.c_attn = nn.Linear(                   PT  # fused Q+K+V — all heads at once
            config.n_embd, 3 * config.n_embd, bias=False
        )                                              # (384, 1152)

        self.c_proj = nn.Linear(                   PT  # output: recombines all heads
            config.n_embd, config.n_embd, bias=False
        )                                              # (384, 384)

        self.attn_dropout  = nn.Dropout(config.dropout)  PT
        self.resid_dropout = nn.Dropout(config.dropout)  PT
        self.n_head = config.n_head    # 6           NC
        self.n_embd = config.n_embd    # 384         NC

        self.register_buffer('bias',               PT  # causal mask — not a parameter
            torch.tril(torch.ones(config.block_size, config.block_size))
            .view(1, 1, config.block_size, config.block_size)
        )
        # bias shape: (1, 1, 1024, 1024)
        # (1, 1, ...) broadcasts across B and n_head
        # 1s below diagonal = allowed to attend
        # 0s above diagonal = masked to -inf before softmax
```

★ The Complete Dimension Trace

Every shape change from raw token IDs all the way to attention scores — nothing skipped. The highlighted row is the dimension that is **new** at each step.

Step

Operation

Shape

Notes

0

get_batch() NC

(B, T)

(12, 1024) int64 — raw token IDs from train.bin

1

wte(idx) PT

(B, T, C)

(12, 1024, 384) — C appears! 2D → 3D. Each integer → 384 floats
2
+ wpe(pos) PT
(B, T, C)
(12, 1024, 384) — shape unchanged, position added via broadcasting
3
c_attn(x) PT
(B, T, 3C)
(12, 1024, 1152) — Q+K+V fused. C triples temporarily
4
.split(384, dim=2) PT
q,k,v: (B, T, C)
(12, 1024, 384) × 3 tensors — separated but heads still mixed in C
5
.view(B,T,6,64) PT
(B, T, n_h, d_h)
(12, 1024, 6, 64) — 4D! Heads labelled. Zero data copy.
6
.transpose(1,2) PT
(B, n_h, T, d_h)
(12, 6, 1024, 64) — heads moved to dim 1 for batched matmul
7 ★
q @ k.T(-2,-1) PT
(B, n_h, T, T)
(12, 6, 1024, 1024) — 64 cancels. 75M scores! One per token pair per head.

Three moments a new dimension appears

Step 1: wte adds C=384 → tensor goes 2D→3D · Step 5: .view() adds n_head and head_dim → tensor goes 3D→4D · Step 7: matmul adds second T → 64 dims cancel and become T×T scores

Attention is quadratic in T — the fundamental bottleneck

The (T, T) score matrix grows with T^2. At T=1024: 75,497,472 numbers just for raw scores (before mask and softmax). Double the context window → 4× the attention memory. This is why context window length is the primary VRAM bottleneck in LLM training and why long-context models are expensive.

Reference

PyTorch Built-ins Cheat Sheet

Every [PT] built-in used in nanochat — one line each

API	WHAT IT DOES	PHASE
torch.tensor(data, dtype=)	Convert Python list/array to tensor	1

API	WHAT IT DOES	PHASE
torch.randint(high, size)	Random integers — batch sampling start positions	1
torch.stack([t1, t2, …])	Stack list of 1D tensors into a 2D matrix	1
torch.from_numpy(arr)	Convert numpy array to tensor (shares memory)	1
.to(device)	Move tensor to CPU or GPU	1
nn.Embedding(num, dim)	Lookup table: integer → row of weight matrix. No multiply.	2
nn.Linear(in, out, bias=)	Fully-connected layer: y = $xW^T + b$	2
nn.Dropout(p)	Zero random p fraction of values during training. Auto-off at eval.	2
nn.ModuleDict(dict)	Named dict of layers — all tracked as trainable parameters	2
nn.ModuleList([…])	Ordered list of layers — all tracked as trainable parameters	2
nn.LayerNorm(dim)	Normalise to mean=0, std=1 per vector. NOT ±1 clipping.	2
nn.GELU()	Smooth activation — like ReLU but fades near 0. Better gradients.	2
torch.arange(n, device=)	Create tensor [0, 1, 2, …, n-1]. Used for position indices.	2
torch.optim.AdamW(…)	Optimiser: Adam + decoupled weight decay. Standard for LLMs.	2
clip_grad_norm_(params, max)	Cap gradient vector magnitude. Prevents catastrophic updates.	2
F.cross_entropy(logits, tgts)	Loss: log-softmax + negate, fused and numerically stable.	Arch
F.softmax(x, dim=)	Convert logits to probabilities summing to exactly 1.0	Arch
probs.argmax(dim=)	Index of highest value — greedy deterministic decoding	Arch
torch.multinomial(probs, n)	Sample index weighted by probabilities — creative generation	Arch
torch.cat([t1, t2], dim=)	Concatenate tensors along a dimension	Arch
torch.tril(ones(T, T))	Lower triangular matrix of 1s — the causal mask	Arch
register_buffer(name, tensor)	Save tensor with model but not as a trainable parameter	Arch
tensor.view(*shape)	Reshape without copying. Relabels memory layout. 3D→4D for heads.	3.1
tensor.transpose(dim_a, dim_b)	Swap two dimensions. Moves heads from dim 2 to dim 1.	3.1
tensor.split(size, dim)	Split tensor into equal chunks. Splits 1152 → q, k, v of 384 each.	3.1
@ / torch.matmul(a, b)	Batched matrix multiply. All B×n_head pairs run in parallel on GPU.	3.1
torch.no_grad()	Disable gradient tracking. Faster inference and eval.	5
model.train() / model.eval()	Toggle dropout on/off. Always call eval() before generating.	5
torch.save(obj, path)	Save checkpoint dict to disk	5
torch.load(path, map_location=)	Load checkpoint. map_location='cpu' handles GPU→CPU.	5

API	WHAT IT DOES	PHASE
torch.amp.autocast(dtype=)	Mixed precision — run forward in bfloat16 for 2-3× speedup.	5
torch.compile(model)	JIT compile via Triton. 30-50% speedup. PyTorch 2.0+.	5

nanochat Learning Journal

Phase 1 — Tokenisation · Phase 2 — Embeddings · Phase 3.1 — Q, K, V Projections

Phases 3.2 → 5 coming: attention scores · causal mask · multi-head · transformer block · training · generation

APPENDIX

Learning Journal — Phase 1

Running notes from the very first build session. Preserved so you can see the thinking, not just the polished outcome.

> Use this when you want the 'how did we decide that?' rather than 'what's the final answer?'

Building an LLM from scratch — concepts, code, and intuition

> ***How to use this document*** *Each phase maps to the nanochat build roadmap. Every code block is annotated: -* `# [PT]` *= PyTorch built-in — comes from the library, just use it -* `# [NC]` *= nanochat custom — Karpathy wrote it, understand every line*
> *Shape traces show tensor dimensions at each step:* `(B, T) → (B, T, C)` *where B = batch size, T = sequence length (time), C = channels (embedding dim)*

PyTorch Built-ins Quick Reference

Populated as we go — one entry per new built-in encountered

API	WHAT IT DOES	FIRST SEEN
`torch.tensor(data, dtype=)`	Convert Python list/array to tensor	Phase 1
`torch.randint(high, size)`	Random integers — used for batch sampling	Phase 1
`torch.stack([t1, t2, ...])`	Stack list of 1D tensors into 2D matrix	Phase 1
`torch.from_numpy(arr)`	Convert numpy array to tensor (shares memory)	Phase 1
`.to(device)`	Move tensor to CPU or GPU	Phase 1
`nn.Embedding(num, dim)`	Lookup table: integer → dense vector	Phase 1 (referenced)
`nn.Linear(in, out)`	Fully connected layer: $y = xW^T + b$	Phase 1 (referenced)

Phase 1 — Tokenisation

How raw text becomes integer tensors the model can process

The problem tokenisation solves

A neural network can only work with numbers. Raw text must become a sequence of integers before any matrix multiplication can happen. The tokeniser is the bridge — it sits entirely outside the PyTorch model, has no trainable parameters, and runs once before training.

```
"Hello world"  →  [15496, 995]  →  nn.Embedding  →  (2, 384) float tensor
    text           token IDs          [PT]            model input
```

Three vocabulary strategies exist, each a trade-off between vocabulary size and tokens-per-word:

STRATEGY	VOCAB SIZE	TOKENS FOR "UNHAPPINESS"	USED IN
Character-level	65 (Shakespeare)	11 (one per char)	nanogpt demo
BPE subword	50,257	3 (`un` , `happiness` , end)	nanochat / GPT-2
Word-level	millions	1	almost never

The context window trade-off: character-level uses ~4× more tokens than BPE for the same text. A 1024-token context window holds ~930 characters at char-level vs ~4000 characters at BPE. This is the primary reason real LLMs use BPE.

1.1 — BPE (Byte Pair Encoding)

THE CORE IDEA

BPE is greedy statistics, nothing more. Start with individual characters, repeatedly find the most frequent adjacent pair in your corpus, merge it into a new token, repeat until you hit your target vocabulary size. The ordered list of merge rules that results IS the tokeniser.

THE ALGORITHM STEP BY STEP

Starting corpus (with end-of-word marker `Ġ`):

```
"low"    → l o w Ġ
"lower"  → l o w e r Ġ
"newest" → n e w e s t Ġ
```

Initial vocabulary: just the 9 unique characters: `l o w e r n s t Ġ`

Merge 1: Count all adjacent pairs. `l+o` appears twice (in "low", "lower") — wins.

```
l o w Ġ      →  lo w Ġ
l o w e r Ġ  →  lo w e r Ġ
n e w e s t Ġ→  n e w e s t Ġ   (unchanged)
```

Vocabulary grows to 10: adds `lo`

Merge 2: `lo+w` now appears twice — wins.

```
lo w Ġ      →  low Ġ
lo w e r Ġ  →  low e r Ġ
```

Vocabulary grows to 11: adds `low`

After many merges, `"lowering"` (a word never seen in training) still tokenises cleanly:

```
"lowering" → low + er + ing + Ġ
```

`"low"` is shared across "low", "lower", "lowering", "lowest" — their embeddings start from the same token. Meaning is shared by construction. This is why BPE beats word-level vocabulary.

FROM-SCRATCH PYTHON IMPLEMENTATION

```python
# [NC] All code below is nanochat custom — plain Python, no PyTorch
from collections import Counter

def get_pairs(vocab):
    """Count all adjacent pairs across the whole corpus."""
    pairs = Counter()
    for word, freq in vocab.items():
        symbols = word.split()
        for i in range(len(symbols) - 1):
            pairs[(symbols[i], symbols[i+1])] += freq
    return pairs

def merge_vocab(pair, vocab):
    """Replace all occurrences of the winning pair with the merged token."""
    new_vocab = {}
    bigram = ' '.join(pair)
    replacement = ''.join(pair)
    for word in vocab:
        new_word = word.replace(bigram, replacement)
        new_vocab[new_word] = vocab[word]
    return new_vocab

# Initial vocab: each word as space-separated chars, with frequency
vocab = {
    'l o w Ġ':      5,   # "low" appears 5 times in corpus
    'l o w e r Ġ':  2,
    'n e w e s t Ġ': 6,
    'w i d e r Ġ':   3,
}

num_merges  = 10
merge_rules = []          # this ordered list IS the tokeniser

for i in range(num_merges):
    pairs = get_pairs(vocab)
    best  = max(pairs, key=pairs.get)   # most frequent pair
    vocab = merge_vocab(best, vocab)
    merge_rules.append(best)
    print(f"Merge {i+1}: {best[0]!r} + {best[1]!r} → {''.join(best)!r}")

# Encoding new text: apply merge rules in order
def tokenise(text, merge_rules):
    words  = text.split()
    tokens = [' '.join(list(w)) + ' Ġ' for w in words]
    for (a, b) in merge_rules:
        tokens = [t.replace(f'{a} {b}', f'{a}{b}') for t in tokens]
    result = []
    for t in tokens:
        result.extend(t.split())
    return result

tokenise("low lower", merge_rules)
```

```
# → ['lowĠ', 'low', 'erĠ']
#    ↑ complete word   ↑ two tokens sharing 'low'
```

Key insight: `merge_rules` is a plain Python list. Serialise it to disk and you have a complete, reloadable tokeniser — no PyTorch needed.

1.2 — Character-level tokeniser (nanogpt Shakespeare demo)

The simplest possible tokeniser. Vocabulary = every unique character in your training text. No merge rules, no external dependencies — just two Python dicts.

THE COMPLETE IMPLEMENTATION

```
import torch                        # [PT] only needed for tensor conversion

# ── Step 1: build vocab ──────────────────────────────────────
with open('shakespeare.txt', 'r', encoding='utf-8') as f:
    text = f.read()                 # [NC]

chars     = sorted(list(set(text)))   # [NC] sorted = deterministic mapping
vocab_size = len(chars)               # [NC] 65 for Shakespeare
# sorted() is critical: without it, set() gives random order each run,
# making saved model weights map to wrong characters on reload

# ── Step 2: lookup tables ────────────────────────────────────
stoi = { ch: i for i, ch in enumerate(chars) }   # [NC] string → int
itos = { i: ch for i, ch in enumerate(chars) }   # [NC] int → string
# stoi['h'] → 34    (integer ID for 'h')
# itos[34]  → 'h'   (back to the character)

# ── Step 3: encode / decode ──────────────────────────────────
encode = lambda s: [stoi[c] for c in s]           # [NC] str  → list[int]
decode = lambda l: ''.join([itos[i] for i in l])   # [NC] list[int] → str

# round-trip always works:
decode(encode("Hello")) == "Hello"   # True

# ── Step 4: encode entire corpus → tensor ────────────────────
data = torch.tensor(encode(text), dtype=torch.long)
#      ^^^^^^^^^^^^                 ^^^^^^^^^^^
#      [PT] creates tensor          [PT] long = int64 (required by nn.Embedding)
#                    ^^^^^^^^^^^^^^
#                    [NC] our encode() from step 3
# shape: torch.Size([1115394])  ← 1.1M characters

# ── Step 5: train / val split ────────────────────────────────
n          = int(0.9 * len(data))   # [NC]
train_data = data[:n]               # [NC]
val_data   = data[n:]               # [NC]

# ── Step 6: batch sampler ────────────────────────────────────
block_size = 256   # [NC] context length — chars the model sees at once
batch_size = 64    # [NC] independent sequences per batch

def get_batch(split):                                          # [NC]
    d  = train_data if split == 'train' else val_data
    ix = torch.randint(len(d) - block_size, (batch_size,))   # [PT]
    x  = torch.stack([d[i  :i+block_size  ] for i in ix])    # [PT]
    y  = torch.stack([d[i+1:i+block_size+1] for i in ix])    # [PT]
    return x.to(device), y.to(device)                          # [PT]

xb, yb = get_batch('train')
# xb shape: (64, 256)  — integer tensor, each cell is a char ID
# yb shape: (64, 256)  — same, shifted right by 1 (the targets)
```

WHAT X AND Y LOOK LIKE (BLOCK_SIZE=8 FOR CLARITY)

```
x:  [18, 47, 56, 57, 58,  1, 15, 47]   →  "First Ci"
y:  [47, 56, 57, 58,  1, 15, 47, 58]   →  "irst Cit"

Position 0 says: "given F,       predict i"
Position 1 says: "given Fi,      predict r"
Position 2 says: "given Fir,     predict s"
...
Position 7 says: "given First Ci, predict t"

One sequence of length 256 gives 256 training examples — free!
```

PYTORCH BUILT-INS USED (ALL IN GET_BATCH)

CALL	WHAT IT DOES
`torch.randint(high, size)`	Randomly picks `batch_size` start positions
`torch.stack([...])`	Stacks list of 1D slices into `(B, T)` matrix
`.to(device)`	Moves tensor to GPU if `device='cuda'`

Everything else — `set()`, `sorted()`, `enumerate()`, `lambda`, `dict` — is plain Python with zero PyTorch.

1.3 — tiktoken (what nanochat actually uses)

tiktoken is not a new algorithm. It is GPT-2's pre-trained BPE tokeniser — 50,000 merge rules trained by OpenAI on web text, shipped as a fast Rust library. The interface mirrors your char tokeniser exactly: `encode()` and `decode()`.

WHY TIKTOKEN OVER CHAR-LEVEL

	CHAR TOKENISER	TIKTOKEN GPT-2
Vocab size	65	50,257
Chars per token	1	~4
Context window efficiency	poor	4× better
Vocab trained on	your corpus	OpenAI web text
External dependency	none	`pip install tiktoken`

BASIC USAGE

```python
import tiktoken                                  # [NC]

enc = tiktoken.get_encoding("gpt2")              # [NC]
# "gpt2"         → 50,257 tokens (nanochat uses this)
# "cl100k_base"  → 100,277 tokens (GPT-4)
# Downloads ~500KB merge rules once, caches in ~/.tiktoken

print(enc.n_vocab)     # 50257
print(enc.eot_token)   # 50256 ← special end-of-text token ID

# encode: string → list of ints
ids = enc.encode("Hello, I'm a language model")   # [NC]
# → [15496, 11, 314, 1101, 257, 3303, 2746]
```

```
# decode: list of ints → string
enc.decode(ids)                                  # [NC]
# → "Hello, I'm a language model"

# Inspect individual tokens — very useful for debugging:
for tok_id in ids:
    print(f"{tok_id:6d}  →  {repr(enc.decode([tok_id]))}")
# → 15496  →  'Hello'
# →    11  →  ','
# →   314  →  ' I'     ← note the leading space!
# →  1101  →  "'m"
# →   257  →  ' a'
# →  3303  →  ' language'
# →  2746  →  ' model'
```

> ***Spaces are part of tokens.*** `' I'` *(space + I) is one token. GPT-2's BPE was trained on text where spaces attach to the following word. Always use the same tokeniser your model was trained with — the space convention is baked into the vocabulary.*

ENCODE_ORDINARY VS ENCODE — IMPORTANT DISTINCTION

```
# encode() raises ValueError if special tokens appear, unless allowed:
enc.encode("hello <|endoftext|> world",
           allowed_special={"<|endoftext|>"})
# → [31373, 220, 50256, 995]
#              ↑ 50256 = EOT special token ID

# encode_ordinary() — ignores special tokens, treats them as text.
# THIS is what nanochat's prepare.py uses for raw data:
enc.encode_ordinary("hello <|endoftext|> world")  # [NC]
# → [31373, 1279, 91, 437, 1659, 5239, 91, 29, 995]
#    treats <> as regular characters

# nanochat adds EOT between documents manually:
eot    = enc.eot_token                           # [NC] = 50256
doc1   = enc.encode_ordinary("First document...")
doc2   = enc.encode_ordinary("Second document...")
tokens = doc1 + [eot] + doc2                     # [NC]
```

DATA PREPARATION — PREPARE.PY (RUN ONCE BEFORE TRAINING)

```
# prepare.py — run ONCE before training, never again           [NC]
import os, tiktoken
import numpy as np

with open('data/input.txt', 'r', encoding='utf-8') as f:
    data = f.read()

enc = tiktoken.get_encoding("gpt2")              # [NC]
ids = enc.encode_ordinary(data)                  # [NC]
# 1M chars ÷ ~4 chars/token ≈ 250K tokens

n        = int(0.9 * len(ids))
train_ids = ids[:n]
val_ids   = ids[n:]

# Save as uint16 — token IDs 0..50256 fit in 2 bytes
# uint16 = 2 bytes/token vs int64 = 8 bytes/token → 4× smaller files
np.array(train_ids, dtype=np.uint16).tofile('data/train.bin')
np.array(val_ids,   dtype=np.uint16).tofile('data/val.bin')
```

LOADING IN THE TRAINING LOOP — GET_BATCH WITH MEMMAP

```
import torch, numpy as np
```

```
# Memory-map — OS loads file pages on demand, doesn't load all into RAM
train_data = np.memmap('data/train.bin', dtype=np.uint16, mode='r')  # [NC]
val_data   = np.memmap('data/val.bin',   dtype=np.uint16, mode='r')  # [NC]

block_size = 1024   # [NC] GPT-2 context length
batch_size = 12     # [NC] tune to your GPU VRAM

def get_batch(split):                                          # [NC]
    data = train_data if split == 'train' else val_data
    ix   = torch.randint(len(data) - block_size, (batch_size,))  # [PT]
    x = torch.stack(                                           # [PT]
        [torch.from_numpy(                                     # [PT]
            data[i  :i+block_size  ].astype(np.int64)
         ) for i in ix]
    )
    y = torch.stack(                                           # [PT]
        [torch.from_numpy(                                     # [PT]
            data[i+1:i+block_size+1].astype(np.int64)
         ) for i in ix]
    )
    return x.to(device), y.to(device)                          # [PT]

xb, yb = get_batch('train')
# xb: torch.Size([12, 1024])  dtype=torch.int64
# yb: torch.Size([12, 1024])  dtype=torch.int64
```

> ***Why `.astype(np.int64)` in get_batch?*** *Files are stored as uint16 (compact). `nn.Embedding` requires int64. The conversion happens here — compact on disk, correct dtype in the model.*

THE VOCAB_SIZE HANDOFF TO THE MODEL

`vocab_size` is the single number that connects the tokeniser to the model. It determines the size of two things — and only two things:

```
import tiktoken
import torch.nn as nn                              # [PT]
from dataclasses import dataclass

enc = tiktoken.get_encoding("gpt2")

@dataclass
class GPTConfig:                                   # [NC]
    vocab_size : int = enc.n_vocab   # 50257 ← from tiktoken
    block_size : int = 1024          # context length
    n_layer    : int = 6             # transformer blocks
    n_head     : int = 6             # attention heads
    n_embd     : int = 384           # embedding dimension

class GPT(nn.Module):                              # [NC] class
    def __init__(self, config):
        super().__init__()                         # [PT]

        # Place 1 — embedding table
        self.wte = nn.Embedding(config.vocab_size, config.n_embd)  # [PT]
        # 50257 rows × 384 cols = 19.3M parameters
        # Each row = learnable vector for one token

        # Place 2 — output projection
        self.lm_head = nn.Linear(config.n_embd, config.vocab_size, bias=False)  # [PT]
        # 384 → 50257 logits (one score per vocab token)
```

COMPLETE SHAPE FLOW — PHASE 1 OUTPUT INTO THE FULL MODEL

```
get_batch()   →  (B, T)          int64    token IDs from train.bin
wte           →  (B, T, 384)     float32  one vector per token  ← Phase 2
6× Block      →  (B, T, 384)     float32  richer meaning per token
lm_head       →  (B, T, 50257)   float32  one logit per vocab token
```

```
cross_entropy →  scalar loss              compared against y targets

B = batch_size = 12
T = block_size = 1024
384 = n_embd (embedding dimension)
50257 = vocab_size (one output score per possible next token)
```

Phase 1 — Key takeaways

1. **The tokeniser is pure Python** — no PyTorch, no GPU, no training. It runs once on your raw text and produces integer arrays.
2. **Two designs, same interface** — char tokeniser uses `stoi`/`itos` dicts; tiktoken uses compiled Rust. Both expose `encode(str) → list[int]` and `decode(list[int]) → str`.
3. **BPE merge rules are the tokeniser** — not the final vocabulary list. To encode new text, apply each merge rule in order to the character sequence.
4. **vocab_size flows into exactly two model layers** — `nn.Embedding` (input) and `nn.Linear` lm_head (output). Change tokeniser → change both.
5. **dtype discipline** — save as `uint16` (compact), load and convert to `int64` in `get_batch()` (required by `nn.Embedding`).
6. **PyTorch built-ins in Phase 1** — only in `get_batch()`: `torch.randint`, `torch.stack`, `torch.from_numpy`, `.to(device)`, `torch.tensor`

Next: Phase 2 — Embeddings. How each integer ID becomes a 384-dimensional float vector, and how the model learns word order.